UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II

Facoltà di Ingegneria
Corso di Studi in Ingegneria Informatica

tesi di laurea

Un'applicazione per lo scambio di contenuti multimediali in ambiente Android

Anno Accademico 2009/2010

relatore
Ch.mo prof. Simon Pietro Romano

correlatore
Ing. Lorenzo Miniero
Ing. Alessandro Amirante

candidato
Alessandro Arrichiello
matr. 534/002442

Alla mia famiglia..

Ringraziamenti

Vorrei ringraziare innanzitutto i miei genitori che mi hanno supportato moralmente e economicamente in tutto il periodo universitario, permettendomi di arrivare a tale traguardo.

Un immenso grazie va alla mia fidanzata Tecla che ha sopportato i miei repentini cambi d'umore “da stress” prima di ogni esame e mi ha sempre sostenuto lungo tutto il tragitto percorso.

Grazie anche a mio fratello Gianluca che dopo tutte le volte che l'ho mandato a quel paese continua a darmi retta (dopotutto anche lui mi ci manda spesso).

Un gigantesco grazie va poi a tutto il gruppo del laboratorio ARCLAB di cui cito i nomi in ordine rigorosamente decrescente per lunghezza di capelli: prof. Simon Pietro Romano, Lorenzo Miniero, Alessandro Amirante, Tobia Castaldi e tutti gli altri! Ringrazio anche Roberta Presta che mi ha sempre difeso dalle angherie di Alessandro e Tobia.

Ringrazio e mi scuso con l'access-point del laboratorio più volte imputato come colpevole di guasti che non gli appartenevano.

Ringrazio inoltre tutti i miei nonni per avermi continuamente incoraggiato e sostenuto, insegnandomi a non mollare mai e a non pormi mai limiti nella vita.

Ringrazio tutti gli amici e i compagni di studio con cui ho condiviso almeno un po' della mia carriera universitaria: Giacinto, MarcoREDS, Francesco, Mario, Lello, MarcoCompi, Giggin'o'bell, Tony, Pasquale, Enzo, Daviduccio, AnTozzi, Luca e Alberto.

Un particolare ringraziamento va al mio collega Flavio con cui ho condiviso tutti gli ultimi esami e tutto il periodo di tirocinio e tesi.

Ultimo ringraziamento ma non per questo meno importante va allo zio Pascale dell'azienda di pulizie di Agnano che ci ha permesso più volte di restare fino a tardi a studiare.

L'ultimissimo ringraziamento va a Microsoft Office 2008 per Mac OSX che dopo i ripetuti crash mi ha fatto ritrovare e apprezzare le potenzialità della suite open source OpenOffice.

Indice

Introduzione 6

Capitolo 1. Stato dell'arte 9

1.1 VoIP (Voice over IP) 9
1.2 Session Initiation Protocol (SIP) 10
1.3 Piattaforma Meetecho 12
1.4 Potenzialità della piattaforma Android 13
1.4.1 La Dalvik Virtual Machine 14
1.4.2 Android SDK 15
1.4.3 Architettura delle applicazioni in ambiente Android 16
1.4.4 Interfaccia grafica in ambiente Android 17
1.5 La gestione dei dispositivi multimediali in Android 18
1.6 Alternative closed-source 19
1.7 Alternativa Opensource: Sipdroid 19
1.8 Altri protocolli utilizzati: XMPP, BFCP 21

Capitolo 2. Progettazione 23

2.1 Specifica ed analisi dei requisiti 23
2.1.1 Requisiti Funzionali 24
2.1.2 Requisiti Operativi 24
2.2 Progettazione dell'engine del client 26
2.2.1 Interfaccia Grafica 27
2.2.2 UserAgent 28
2.2.3 Modulo SIP 29
2.2.4 Modulo gestione media 30
2.2.5 Modulo Riproduzione/Registrazione Audio 31
2.2.6 Modulo SDP Parser 32
2.2.7 Modulo RTP 33
2.2.8 Modulo BFCP 34
2.29 Modulo CCMP 35

Capitolo 3. Implementazione 36

3.1 Scelte implementative 36
3.2 Diagramma delle classi implementate 37
3.3 Implementazione modulo SIP 38
3.4 Implementazione modulo SDP Parser 41
3.5 Implementazione modulo RTP 43
3.6 Implementazione GestioneMedia e Registrazione/Riproduzione Audio 44
3.7 Implementazione modulo BFCP 48
3.8 Implementazione modulo UserAgent 51
3.9 Implementazione dell'interfaccia grafica 63

Capitolo 4. Sperimentazione 72

4.1 Emulatore di ambiente Android 72
4.2 Android Debug Bridge (ADB) 73
4.3 Differenze tra Emulatore e dispositivo reale 74
4.4 Testing dell'applicativo 75
4.4.1 Avvio della comunicazione 76
4.4.2 Richiesta risorsa tramite BFCP 79
4.4.3 Conferenza in pausa 80
4.4.4 Riproduzione Audio in vivavoce 81
4.4.5 Menu moderatore 82
4.4.6 Interruzione della comunicazione 84
4.5 Testing sul dispositivo reale 85
4.6 Testing delle prestazioni 85
4.7 Problemi riscontrati 87
4.7.1 Soluzione con l'uso della classe nativa `Mediaplayer` 88
4.7.2 Soluzione con l'uso di una implementazione Java del codec H.263 89
4.7.3 Soluzione con l'uso di un server RTSP 90

Conclusioni e Sviluppi Futuri 92
Bibliografia 95

Introduzione

La crescente diffusione e l'abbassamento dei costi delle connessioni Internet per dispositivi mobili hanno portato allo sviluppo di applicazioni multimediali capaci di fornire contemporaneamente i servizi di audio, video e dati in modalità *real-time*.

Tra questi servizi si è notevolmente diffuso il *conferencing*.

Questo complesso servizio multimediale si è sviluppato soprattutto grazie all'introduzione ed alla ricerca, a partire dalla metà degli anni '90, della tecnologia *Voice over IP* (VoIP).

La rete VoIP permette non solo di effettuare conversazioni telefoniche su reti a commutazione di pacchetto mediante il protocollo IP, ma anche una facile convergenza e cooperazione con dati di tutt'altro tipo come possono essere flussi video, messaggi di testo e così via. Questa soluzione porta un notevole risparmio di risorse ed una maggiore scalabilità ma, contemporaneamente, richiede anche la ricerca intensiva di nuovi standard per i servizi multimediali e di soluzioni legate alla qualità del servizio. Ad esempio, le attuali applicazioni nel mondo reale della telefonia VoIP si trovano a dover affrontare problematiche legate a problemi di *latenza* (sostanzialmente si deve ridurre il tempo di transito e di elaborazione dei dati durante le conversazioni) e di *integrità dei dati* (prevenire perdite e danneggiamenti delle informazioni contenute nei pacchetti per mantenere la corretta coerenza temporale dei pacchetti stessi).

La tecnologia VoIP richiede dunque un protocollo di comunicazione per il trasporto "affidabile" dei dati (pacchetti voce su IP) che, nella grande maggioranza delle implementazioni, è il protocollo RTP (*Real-time Transport Protocol*); inoltre consente ampi margini di interoperabilità, mediante opportune piattaforme di transcodifica, fra protocolli di *signaling* come ad esempio SIP (*Session Initiation Protocol*) ed ambienti fra loro completamente diversi.

Come è facilmente intuibile, la presenza di molteplici soluzioni poneva diverse annose questioni come: l'interoperabilità, la trasparenza delle applicazioni rispetto alle piattaforme proprietarie su cui erano state sviluppate, la sicurezza e la qualità del servizio

fruibile in uno scenario di mercato fin troppo frammentato. Per superare questi ostacoli si iniziò un forte studio sulla standardizzazione del servizio di conferencing da parte di diversi enti quali: ITU (*International Telecommunication Union*), ETSI (*European Telecommunications Standards Istitute*), IETF (*Internet Engineering Task Force*) e 3GPP (*3rd Generation Partnership Project*).

Questo lavoro tesi si è concentrato sulle possibili soluzioni di conferencing in ambiente Android, un sistema operativo sviluppato dall'OHA (*Open Handset Alliance*) a partire dal novembre 2007, completamente opensource, giunto alla release *2.2*.

Android risulta essere una piattaforma ancora in fase di forte evoluzione e sviluppo ma, ad oggi, è già in grado di fornire il framework e gli strumenti necessari (tra cui un versatile emulatore) per lo sviluppo e la ricerca di molteplici tipi di applicazioni e servizi multimediali, attraverso l'uso delle API (*Application Programming Interface*) standard di Java ed il linguaggio XML (*eXtensible Markup Language*). La possibilità di utilizzare una piattaforma aperta, consente di poter sviluppare un'applicazione trasparente rispetto alla piattaforma stessa, aiutando notevolmente il lavoro di convergenza, integrazione e sviluppo comunitario.

Si sono quindi dapprima appresi il funzionamento, le potenzialità e le problematiche legate all'ambiente Android per poi sviluppare un client SIP (*Signaling Initiation Protocol*) per il conferencing con l'ausilio del protocollo di moderazione BFCP (*Binary Floor Control Protocol*) e del protocollo di trasporto RTP (*Realtime Transport Protocol*), integrando un client XMPP (*eXtensible Messaging and Prensence Protocol*) per la chat e la condivisione file, frutto di un lavoro precedente.

Si sono poi affrontati problematiche relative ad integrazione di librerie preesistenti ma per ambienti differenti e limitazioni delle API del dispositivo stesso.

Si è inoltre testato l'applicativo su dispositivi reali differenti affrontando limitazioni e bug dovuti alla versione specifica dei produttori di dispositivi mobili cercando di rendere il software allo stesso tempo standard e funzionante su tutti i dispositivi Android.

Il lavoro si è focalizzato sull'invio e ricezione audio nonché sul video, dove però restano delle forti limitazioni dovute alle API specifiche fornite dal sistema stesso, come verrà

spiegato successivamente.

Nel primo capitolo si descriveranno tutti gli aspetti legati all'ambiente di sviluppo, a partire dalla struttura di un'applicazione Android, la gestione delle interfacce grafiche, i servizi disponibili e quelli implementabili per la gestione dei media e dei protocolli SIP, RTP e l'integrazione del protocollo BFCP.

Nel secondo capitolo saranno descritti i requisiti operativi e funzionali e la progettazione dei singoli moduli componenti l'applicativo da sviluppare, supporto della progettazione saranno inoltre forniti diagrammi dei moduli per facilitare la comprensione del lavoro effettuato.

Nel terzo capitolo, si esporrà il lavoro svolto, mostrando come è stato implementato il client attraverso spezzoni di codice, diagrammi della classi e di sequenza.

Nel quarto capitolo, saranno esposti gli scenari tipici in cui è possibile utilizzare l'applicativo sviluppato e la relativa sperimentazione effettuata.

Infine saranno proposte le conclusioni e gli sviluppi futuri sul lavoro di tesi svolto, effettuando un riepilogo generale del progetto con un occhio alle possibili estensioni.

Capitolo 1

Stato dell'arte

Negli ultimi anni, di pari passo con l'evoluzione delle tecnologie VoIP, si è manifestato un crescente interesse al servizio di *conferencing* che ha condotto, grazie ad un notevole sforzo in termini di ricerca, alla progettazione ed alla realizzazione di diverse soluzioni per dispositivi mobili. La diffusione di dispositivi mobili con grandi capacità multimediali ha permesso inoltre che queste tecnologia approdassero con facilità su piattaforme mobili. Grazie all'elevata capacità elaborativa di dispositivi di ultima generazione quali Apple iPhone, dispositivi con sistema Android, Blackberry e SymbianOS, le *software houses* non hanno tardato a rilasciare applicativi che implementassero la tecnologia voip e ogni suo derivato, permettendo lo scambio di contenuti multimediali che estendessero i classici concetti di telefonata o SMS (*Short Message Service*).
Nello specifico per dispositivi Android troviamo diverse alternative *opensource* e non, che danno la possibilità all'utente finale di inviare/ricevere audio verso un contatto registrato allo stesso servizio.Tra queste troviamo i noti applicativi disponibili sulla maggior parte dei dispositivi mobili, quali: Fring, Nimbuzz, Skype, tutti *closed source* e utilizzanti protocolli proprietari, tra gli applicativi *open source* troviamo SipDroid usante protocolli SIP per il *signaling* e RTP per il trasporto dati. E' proprio su quest'ultimo che si è focalizzato lo studio iniziale di questo lavoro di tesi.

1.1 Voice over IP (VoIP)

Voice over IP, acronimo VoIP, è una tecnologia che consente di effettuare una conversazione telefonica sfruttando una connessione Internet, o un'altra rete che utilizza il protocollo IP. Più specificatamente con VoIP si intende l'insieme dei protocolli di comunicazione che permettono tale conversazione. Il vantaggio principale di questa tecnologia è quello di sfruttare l'allocazione dinamica del protocollo IP, dando luogo quindi ad una commutazione di pacchetto, risolvendo quindi il problema di assegnare

obbligatoriamente parte della banda disponibile per ogni telefonata effettuata. Quindi sulla rete vengono instradati pacchetti di dati contenenti informazioni vocali, codificati in formato digitale, e ciò solo quando è necessario quindi solo quando uno degli utenti collegati sta parlando. Oggi grazie ad alcuni provider è possibile effettuare telefonate anche verso la rete telefonica tradizionale Public Switched Telephone Network (PSTN), rendendo quindi questa tecnologia un punto di forza per sviluppi di applicazioni future.

1.2 Session Initiation Protocol (SIP)

Il protocollo SIP (*Session Initiation Protocol*) è un protocollo basato su IP standardizzato dall'IETF nella RFC 3261 e impiegato principalmente per applicazioni di telefonia su IP o VoIP.

SIP gestisce in modo generale una sessione di comunicazione tra due o più entità, ovvero fornisce meccanismi per instaurare, modificare e terminare (rilasciare) una sessione. Attraverso il protocollo SIP possono essere trasferiti dati di diverso tipo (audio, video, messaggistica testuale, ecc). Inoltre, SIP favorisce un'architettura modulare e scalabile, ovvero capace di crescere con il numero degli utilizzatori del servizio. Queste potenzialità hanno fatto sì che il SIP sia, oggi, il protocollo VoIP più diffuso nel mercato residenziale e business, sorpassando di molto altri protocolli quali H.323 ed MGCP.

Il modello usato per la sintassi del protocollo SIP è *text-based*, derivato dall' HTTP. Per instaurare una sessione, avviene un *three-way handshaking* (concettualmente simile a quello che avviene con il protocollo TCP) per la negoziazione dei *codec* da utilizzare per la trasmissione dei dati multimediali, utilizzando il protocollo SDP (*Session Description Protocol*).

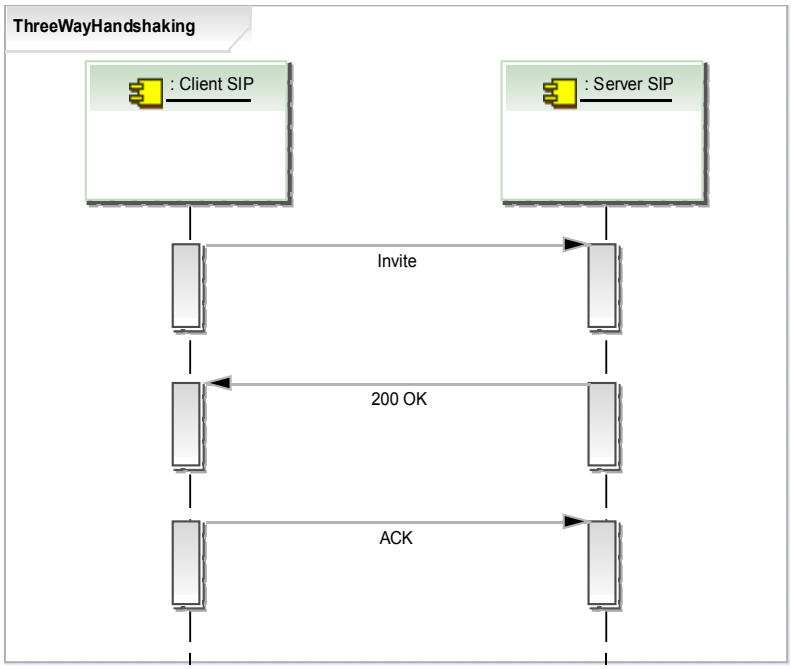


Figura 1.1: Three-Way-Handshaking SIP

Inoltre è impiegabile sia in contesti *client-server* sia in contesti *peer to peer*, è facilmente estendibile, programmabile ed è indipendente dal protocollo di trasporto.

Una sessione SIP è composta da transazioni, una transazione è un insieme di messaggi, identificabile da un transaction-ID, un identificativo che ne specifica la sorgente, la destinazione e il numero di sequenza.

Questo tipo di protocollo oltre che nelle ambito della telefonia su IP si sta però sempre più diffondendo anche negli applicativi di *conferencing* garantendo prestazioni elevate.

Grazie alle specifiche definite dall'IETF ed alle molteplici librerie preesistenti in rete è possibile inoltre portare questo protocollo su dispositivi mobili senza particolari difficoltà.

Il protocollo SIP è affiancato solitamente dal protocollo SDP che si occupa della descrizione dei media che saranno utilizzati nel trasporto dei dati multimediali, questo protocollo nasce come componente di SAP (*Session Announcement Protocol*) ma tutt'oggi è utilizzato in congiunzione ad altri protocolli quali: RTP (*Real-time Transport Protocol*), RTSP (*Real-time Streaming Protocol*) e lo stesso SIP.

Unendo poi SIP e SDP al protocollo di trasporto RTP si riesce ad ottenere una trasmissione con compensazione del *jitter* e rilevazione di pacchetti fuori sequenza, comuni durante le trasmissioni su una rete IP.

Storicamente progettato per funzionare su multicast, il protocollo RTP è principalmente

utilizzato sulla base del protocollo di trasmissione UDP (User Datagram Protocol), un protocollo di tipo *connectionless*, che non gestisce il riordinamento dei pacchetti né la ritrasmissione di quelli persi, quindi di minore affidabilità ma in compenso molto rapido ed efficiente per le applicazioni time-sensitive, quali VoIP e streaming audio/video.

1.3 Piattaforma Meetecho

Il client per web *conferencing* che sarà implementato nel lavoro di tesi dovrà essere capace di interagire con la piattaforma Meetecho.

Meetecho è un potente strumento per web *conferencing*, sviluppato da alcuni membri del Team COMICS di UniNa, che mette a disposizione degli utenti molteplici funzionalità come: audio, video, messaggistica istantanea e presenza, lavagna condivisa, presentazione delle slide, condivisione del desktop, controllo remoto, gestione delle conferenze e moderazione. Tutti questi servizi vengono offerti mediante una piattaforma conforme agli standard internazionali in materia di *conferencing* multimediale.

Oltre ai protocolli già esaminati la piattaforma usa il protocollo XMPP (*eXtensible Messaging and Presence Protocol*) per la chat e la condivisione file. XMPP, precedentemente noto come Jabber, è un protocollo basato su XML sviluppato per l'I*stant-Messaging* e la gestione delle presenze.

Tale conformità permette la fruizione del servizio anche da *client* prodotti da terze parti, come ad esempio nel nostro caso il *client* che verrà realizzato per dispositivi Android. I componenti fondamentali della piattaforma sono:

- Server Openfire: è il server XMPP di riferimento, che oltre a gestire le informazioni sulla presenza e i messaggi istantanei scambiati tra gli utenti collegati, offre la possibilità di gestire le informazioni relative alle conferenze attive, lavagne interattive e slides di presentazione.
- Web Server Tomcat: è il WEB-SERVER di riferimento, utilizzato principalmente per fare l'upload di file usati per le presentazioni dando la possibilità di creare immagini utili alle presentazioni.
- Server Asterisk: è il server PBX (*Private Branch eXchange*) usato per il *signaling*

e la comunicazione tra due o più utenti, si occupa principalmente del mixing audio e della gestione delle conferenze. Occupandosi del *signaling* dei media disponibili lato server, svolge un ruolo centrale nel progetto.

- Video Mixer: è il componente che si occupa del mixing video, fornendo attraverso una codifica in *realtime* un video composto a partire dai dati video acquisiti dal client di ogni partecipante alla conferenza.

1.4 Potenzialità della piattaforma Android

Android è un sistema operativo sviluppato dall'OHA (*Open Headset Alliance*) a partire dal novembre 2007, completamente open source, giunto alla release *2.2*.

Android comprende tutto: *Application Layer*, Sistema Operativo, Librerie API e le applicazioni stesse.

Tra le caratteristiche peculiari di Android troviamo:

- Progetto hardware di riferimento che descrive le capacità richieste da un dispositivo mobile per supportare il *software-stack*.
- Sistema Operativo basato su kernel Gnu/Linux con un'interfaccia di basso livello sull'hardware, gestione della memoria e processo di controllo, tutto ottimizzato per i dispositivi mobili.
- Librerie *open source* per lo sviluppo software: SQLite , WebKit, OpenGL, media-manager.
- *Run-time* efficiente per dispositivi mobili usato per le applicazioni, la Dalvik *Virtual Machine* e le librerie specifiche di Android.
- *Application Framework* che espone i servizi di sistema all'*Application Layer*, inclusi il gestore delle finestre, gestore delle *locations*, gestore dei contenuti, telefonia, sensori, etc.
- *Framework* interfaccia utente per ospitare e avviare applicazioni.
- Applicazioni preinstallare fornite come parte dello *stack*.
- *Software* di sviluppo multi-piattaforma.

Le applicazioni native che troviamo incluse nello *stack* di Android possono essere di tipo

proprietario, ma in entrambi i casi è possibile condividere i dati di tali applicazioni con le proprie applicazioni intercettando anche eventi quali: telefonia e messaggistica SMS. Gli OEM possono comunque personalizzare e modificare a loro piacimento le interfacce con delle alternative proprietarie quali: HTC Sense, Motorola MOTOBLUR, etc.

1.4.1 La Dalvik Virtual Machine

Android è stato progettato per essere utilizzato su hardware differente grazie all'ausilio della *Virtual Machine* utilizzata: "Dalvik". La Dalvik *Virtual Machine* è basata su un'architettura a registri a differenza delle altre VM che sono basate su *stack*.

- Le VM *stack-based* devono usare le istruzioni per caricare i dati nello *stack* e manipolare tali dati, questo richiede inoltre più istruzioni rispetto a una VM *register-based* per implementare lo stesso codice di alto livello.
- La VM *register-based* usano istruzioni che devono codificare registri sorgente e destinazione e quindi tendono a diventare più grandi.

L'uso della Dalvik VM non consente l'esecuzione nativa di classi Java compilati e/o eseguibili su Android. Attraverso il tool "dx" le classi *.Java* vengono convertite in formato *.dex*. Più classi Java sono incluse nello stesso file *dex* e i duplicati di stringhe e costanti sono inseriti una sola volte per risparmiare spazio. Un file *dex* compresso ha in genere una grandezza minore rispetto al corrispettivo *jar* compresso, derivato dalla stessa Java class. Gli eseguibili della Dalvik possono ancora essere modificati una volta installati per consentire future ottimizzazioni.

Oltre al fatto di essere stata ottimizzata per dispositivi con poca memoria, la Dalvik Virtual Machine ha alcune caratteristiche specifiche che la distinguono da una Virtual Machine standard:

- La *Virtual Machine* è stata snellita per usare meno spazio possibile
- La Dalvik non ha un *just-in-time compiler*, tuttavia da Android 2.0 è iniziata la sperimentazione di sorgenti sperimentali.
- L'insieme delle costanti è stato modificato per utilizzare un indirizzamento di soli 32-bit per semplificare l'interprete

- Usa il suo stesso *bytecode* e non quello di Java
- E' stata progettata per essere eseguita con più istanze contemporaneamente, ogni applicazione Android viene avviata nella propria istanza della Dalvik VM.
- Non rispetta i profili delle librerie Java SE/ME, ma usa una sua libreria creata a partire dall'implementazione Java: Apache Harmony, un'implementazione Java free sotto licenza Apache 2.0.

1.4.2 Android SDK

Parecchie delle potenzialità del sistema Android risiedono nel suo SDK che non ha nessun costo di licenza, distribuzione o sviluppo, nessun processo di approvazione *software*, accesso *hardware* al wifi, accesso agli eventi delle reti GSM, EDGE, 3G, fornisce inoltre le API per il controllo del GPS, controllo hardware dei dispositivi multimediali, API per accelerometro e *compass*, liberie per il bluetooth, IPC per scambio messaggi, *Data Stores* condivisi, applicazioni e processi in *background*, *widgets*, sfondi animati, ricerca integrabile in quella di sistema, browser HTML5 integrato, pieno supporto ad applicazioni che si basano su mappe, librerie OpenGL, possibilità di sostituzione di applicazioni native.

Come abbiamo già accennato il modulo SDK fornito da Android è multi-piattaforma, utilizzabile in diversi ambienti operativi, a tal proposito è necessario citare il *plugin* di sviluppo fornito per Eclipse installabile in pochi *click*, aggiungendo un sito di *update* remoto ad Eclipse.

Dopo aver installato SDK e plugin, l'ultimo passo sarà quello di avviare l' AVD Manager (*Android Virtual Device Manager*) che permetterà in pochi passaggi di creare un nuovo device virtuale scaricando la versione di Android scelta come target build.

Oltre al tool di sviluppo SDK, Android fornisce anche un NDK (*Native Development Kit*), in grado di fornire un accesso di basso livello al sistema, mentre la Dalvik VM è un livello aggiunto tra sistema Gnu/Linux e Applicazioni, con l'uso del NDK è possibile creare applicazioni eseguibili direttamente dal sistema Gnu/Linux sottostante in C/C++.

La comunicazione inter-applicazione può avvenire con notifiche (audio, vibrazione, led), o con gli *Intents* (passaggio messaggi tra applicazioni), oppure con i *Content providers*

(accesso a *Data Stores* condivisi).

Il supporto multimediale è esteso e tra i formati supportati nativamente si può citare: MPEG4, MP3, AAC, AMR, JPG, PNG, GIF.

Un altro aspetto fondamentale del sistema Android è il suo *runtime*, completamente indipendente dalle applicazioni stesse, può decidere indipendentemente quali processi terminare e quali lasciare in esecuzione, in base a priorità specifiche quali ad esempio il Focus dell'applicazione, l'applicazione possedente il focus infatti ha sempre priorità maggiore e viene preservata a discapito delle altre.

L'Android SDK include inoltre un emulatore di un dispositivo mobile — un dispositivo virtuale utilizzabile su differenti sistemi operativi quali: Microsoft Windows, Gnu/linux, Mac OSX. Tale emulatore sarà analizzato in dettaglio nei capitoli successivi.

1.4.3 Architettura delle applicazioni in ambiente Android

L'architettura di Android incoraggia il riuso abilitando la pubblicazione e la condivisione di: attività, servizi e dati con altre applicazioni utilizzando la sicurezza e le restrizioni desiderate.

L'architettura di ogni applicazione Android si basa principalmente su seguenti gestori:

- *Activity Manager*: che controlla il ciclo di vita delle attività, inclusa la gestione dello stack delle attività.
- Viste (*Views*): Usate per costruire le interfacce utente per le attività
- *Notification Manager*: gestore delle notifiche per comunicare informazioni agli utenti
- *Content Providers*: permette alle apps di condividere dati.
- *Resource Manager*: permette di esportare/importare stringhe e immagini.

Ogni applicazione Android ha un manifesto, un file in formato XML dove sono registrate tutte le proprietà dell'applicazione:

- *VersionCode*: un intero riservato al sistema
- *VersionName*: un valore alfanumerico riservato all'utente
- *uses-SDK*: che indica il suo SDK di riferimento.

- *Support-Screens*: che specifica le risoluzioni supportate.
- *Permissions*: stabilisce di quali permessi necessita l'applicativo per il corretto funzionamento.

I permessi necessari ad ogni applicazione verrano visualizzati in fase di installazione, in modo che l'utente conosca in modo trasparente cosa l'applicativo può e cosa non può fare, garantendo un incremento della sicurezza.

La scrittura del file *Manifest* è semplificata dall'uso del *Manifest-Editor* incluso nel *plugin* per Eclipse che permette la modifica visuale del file senza operare direttamente sul file XML.

1.4.4 Interfaccia grafica in ambiente Android

Un'applicazione in ambiente Android è composta da vari elementi, quelli principali sono:

- *Activity*: il corrispettivo di un *form* che il nostro applicativo mostra al suo utente.
- *Views*: contenitori e gestori dei layouts e dell'interazione con l'utente
- *ViewGroups*: struttura gerarchica di views, usate per creare componenti riusabili.

Gli stati di un'Activity si suddividono in:

- Attiva
- In Pausa
- *Stopped*
- Inattiva

E' possibile gestire il comportamento del nostro applicativo facendo un opportuno *overriding* delle funzioni al cambio di stato di una *Activity*.

Esistono inoltre delle sottoclassi della *Activity* mirate per un servizio specifico, quali: *MapActivtity*, *ListActivity*, *ExpandableListActivity*, *TabActivity* e così via.

Android mette inoltre a disposizione diversi layouts predefiniti quali:

- *FrameLayout*: E' il più semplice, posiziona i figli, uno sopra l'altro seguendo l'ordine di inserimento come una pila a partire dall'angolo sinistro dello schermo.
- *LinearLayout*: Posiziona gli elementi in righe o colonne.
- *RelativeLayout*: E' il più flessibile dato che fa scegliere l'esatta posizione dei figli.

- *TableLayout*: Posizione gli elementi in tabella.
- *Gallery*: *Layout* della galleria, dove gli elementi sono visualizzati in un'unica grande riga orizzontale.

1.5 La gestione dei dispositivi multimediali in Android

Android offre diverse librerie per l'accesso ai dispositivi multimediali di acquisizione e riproduzione. Le librerie documentate per l'acquisizione sono due: `MediaRecorder` e `AudioRecorder`, le librerie documentate per la riproduzione sono invece: `MediaPlayer` e `AudioTrack`.

Prima del rilascio ufficiale della versione di Android 1.5 non era possibile riprodurre e acquisire contenuti multimediali come buffer, ma bisognava necessariamente salvare il contenuto su file e successivamente riprodurlo in caso di ricezione da rete, o leggerlo da file in caso di invio tramite rete. Questa forte limitazione ha condizionato e ostacolato la creazione di applicazioni multimediali *realtime* come quella sviluppata in questo lavoro di tesi, senza l'introduzione delle librerie `AudioRecorder` e `AudioTrack` che gestiscono l'input/output da buffer si sarebbe aggiunto un *delay* dovuto al passaggio sistematico per il *filesystem* durante il processo di acquisizione/riproduzione.

`MediaRecorder` a differenza di `AudioRecord` permette di semplificare il processo di acquisizione permettendo anche l'acquisizione di un eventuale video da una apposita periferica, tuttavia come già detto non supporta la gestione dei buffer ed è quindi necessario salvare il contenuto acquisito in un file, inoltre `MediaRecorder` a differenza di `AudioRecord` non permette la modifica del sampling durante l'acquisizione audio.

Allo stesso modo `MediaPlayer` fornisce una gestione semplificata di eventuali file multimediali riproducendo anche file video, tuttavia risulta inappropriata per la riproduzione di eventuali buffer audio.

Android si sta continuamente evolvendo, i problemi relativi alla gestione dei buffer audio sono stati risolti ma restano ancora particolari restrizioni sull'acquisizione e la riproduzione video che continuano ad essere limitate dal passaggio sistematico per il *file system* durante il processo di acquisizione/riproduzione, questo problema, come vedremo

nei capitoli successivi, può essere in parte superato.

1.6 Alternative closed-source

I programmi multimediali per scambio di contenuti multimediali in ambiente Android sono molteplici. Tra le alternative *closed-source* possiamo citare: Fring, Nimbuzz, Skype. Data la natura closed-source di queste applicazioni e considerato l'uso di protocolli proprietari, questo lavoro di tesi non ha quindi preso spunto da nessuno dei sopracitati *client*, ma si è solo limitato ad un testing generico di tali applicazioni per verificare cosa realmente rendessero a disposizione dell'utente finale.

Da vari test effettuati sui *client* è emerso che sia Fring che Nimbuzz pur fornendo supporto al protocollo SIP, concentrano il servizio da loro offerto sull'Istant Messaging e sul supporto ai protocolli proprietari di Skype, Windows Live Messenger, Gtalk e così via.

1.7 Alternativa Opensource: Sipdroid

Sipdroid è un *client* SIP o*pen source* fornito sotto licenza GPL. L'obiettivo principale degli sviluppatori è di creare un client per ambiente Android in grado di interfacciarsi a qualsiasi provider SIP, garantendo così all'utente finale un risparmio su telefonate locali e internazionali. Le telefonate risultano chiare sia su rete 3G che su WIFI. La caratteristica principale di Sipdroid, che lo contraddistingue da tutte le alternative o*pen source* e non, è l'integrazione con il servizio di telefonia di Android. Dopo essersi loggati, su un server SIP prescelto o su quello consigliato dal team di sviluppo, le telefonate locali e/o internazionali potranno essere deviate e effettuate attraverso il server SIP a cui si è connessi, sfruttando così eventuali offerte e/o riduzioni del servizio utilizzato.

Sipdroid allo stato attuale è giunto alla versione 1.5 beta ed è pienamente compatibile con la totalità dei dispositivi Android 1.6 e 2.1.

Il *client* inoltre attraverso l'uso della libreria jSTUN garantisce il suo funzionamento attraverso le varie strutture NAT preesistenti. STUN è l'acronimo di "*Session Traversal Utilities for NAT*" ed è l'insieme di metodi e protocolli che permettono ad applicazioni real-time di messaggistica istantanea, audio e video di lavorare attraverso NAT (*Network*

Address Translation).

Dal punto di vista multimediale il *client* Sipdroid mette a disposizione diversi tipi di codec audio garantendo all'utente un buon rapporto di qualità/compressione ed una corretta connessione con la maggior parte dei server SIP. Tra i *codec* audio supportati citiamo: PCMA, PCMU, GSM. Tali *codec* non sono nativamente supportati da Sipdroid che li implementa e gestisce all'interno del suo *client*.

La connessione al server SIP prescelto viene effettuata utilizzando la libreria MjSip, sviluppata da Luca Veltri dell'Università di Parma, riadatta dal team di sviluppo di Sipdroid per un corretto funzionamento. MjSip è una libreria fornita per ambienti Java SE e Java ME, che si occupa del processo di autenticazione e signaling verso il server SIP senza però toccare la parte di trasporto dei media, la libreria infatti permette la segnalazione e l'*handshaking* dei *codec* necessari alla creazione dei canali audio/video, permettendo così al client di interfacciarsi correttamente al server SIP.

Il team di sviluppo di Sipdroid quindi, a partire dalla libreria MjSip, ha sviluppato tutta la parte relativa al trasporto dei media, supportando sia il protocollo TCP che UDP, gestendo il processo di acquisizione, codifica e successivo invio così come la ricezione, la decodifica e la riproduzione, garantendo quindi il pieno supporto allo scambio real-time di contenuti multimediali.

Sipdroid ha inoltre un iniziale supporto all'invio e ricezione video, ancora in fase di testing da parte del team di sviluppo.

La parte iniziale di questo lavoro di tesi è stata lo studio di questo *client*, astraendo ove possibile i riferimenti all'interfaccia grafica, molto radicati in ambiente Android, testando e verificando il funzionamento di tale strumento.

1.8 Altri protocolli utilizzati: XMPP, BFCP

Il *client* che sarà sviluppato in questo lavoro di tesi va ad estendere un lavoro precedente che implementa un *client* XMPP (*eXtensible Messaging and Presence Protocol*) interfacciato con la piattaforma Meetecho.

Il protocollo applicativo XMPP è un'insieme di formalismi utili ad effettuare lo scambio di XML tra due nodi della rete al fine di comunicare soprattutto informazioni di presenza e messaggi in tempo reale. L'architettura alla base di XMPP è una rete di client che comunicano attraverso un server XMPP per mettersi in contatto tra loro. I client si limitano a comunicare con il server, che poi provvede a smistare i messaggi tra i vari utenti. Un'importante e piuttosto recente estensione del protocollo XMPP è stata la definizione delle MUC (*Multi-User Chat*) che sono utilizzate per inviare messaggi di testo in broadcast all'interno di una conferenza tra più utenti.

Un'altro protocollo utilizzato, necessario per interfacciarsi con la piattaforma MeetEcho è BFCP (*Binary Floor Control Protocol*).

Il protocollo BFCP si pone come obbiettivo la gestione dei *Floor*, ovvero risorse condivise in un'ambiente di *conferencing* multi-parte, è stato standardizzato dall'IETF con la RFC4582. Il controllo dei *Floor* però non si ferma a BFCP ma va esteso con altre funzioni come il *setup* di una conferenza con i relativi media, la manipolazione delle *policy* di una conferenza e il controllo dei media stessi, tutti realizzati con altri protocolli. BFCP fornisce un supporto e deve quindi sempre essere affiancato ad altri protocolli.

Nel caso in esame BFCP fornisce un supporto per la gestione delle *policy* per l'invio dell'audio e del video in una conferenza, affiancandosi così al protocollo di *signaling* SIP utilizzato dalla piattaforma Meetecho.

Il protocollo BFCP permette inoltre la gestione di *policy* in modalità automatica oppure controllata da un moderatore. Il protocollo inoltre è stato progettato in ambienti con quantità di banda limitata, la codifica binaria utilizzata da BFCP infatti comprende messaggi di piccole dimensioni in modo da ridurre eventuali ritardi di trasmissione. I messaggi scambiati dal protocollo BFCP si dividono in: FloorRequest, FloorRelease,

FloorRequestStatus, ChairAction e sono utilizzati rispettivamente per: richiedere una risorsa, rilasciare una risorsa, chiedere lo stato di una risorsa, eseguire un'azione da moderatore.

La libreria che sarà utilizzata è frutto di un precedente lavoro di tesi ed è stata sviluppata in Java.

Nel caso in esame il protocollo BFCP è usato unitamente al protocollo SIP attraverso l'uso di un re-invite dopo il processo di negoziazione di audio/video dal server che propone una modifica della conferenza aggiungendo i dettagli per la connessione al servizio di moderazione BFCP, come descritto nella RFC4583.

Capitolo 2

Progettazione

In questo capitolo verranno descritti tutti gli strumenti che consentiranno lo sviluppo dell'applicativo. In particolare saranno delineati i requisiti e le specifiche funzionali del progetto. Sulla base di questa analisi si affronterà la progettazione del *client* cercando di tenere separate l'interfaccia utente dal corpo vero e proprio dell'applicazione, tenendo inoltre conto anche dei vincoli posti dal dispositivo hardware.

Gli obiettivi fissati per il *client* prevedono, oltre alla fase di registrazione di un utente presso il server, la possibilità di realizzare scenari di *conferencing*. In particolare oltre all'utilizzo del protocollo XMPP implementato in un lavoro precedente, si è cercato di concentrare l'attenzione sui messaggi di segnalazione, tramite il protocollo SIP, sulla scelta dei *codec* appropriati alla trasmissione audio e sulla moderazione della conferenza. Il lavoro quindi, ha portato alla progettazione di un software che permettesse la registrazione presso un server SIP e l'instaurazione di una chiamata con il relativo scambio di media e con la relativa gestione della moderazione, con *policy* automatiche o con un moderatore umano.

2.1 Specifica ed analisi dei requisiti

I requisiti di un software si posso dividere in requisiti funzionali e in requisiti operativi.

I requisiti funzionali, quelli relativi al client, sono rappresentati dalle operazioni disponibili, dalle interfacce utente e dagli output mostrati. I requisiti operativi, invece, sono quelli che formalmente vengono definiti dal contesto nel quale dovrà operare il software, come ad esempio nel nostro caso i server con cui si dovrà interfacciare il nostro client.

2.1.1 Requisiti Funzionali

Come è stato già accennato, il software frutto del lavoro di tesi in questione andrà ad estendere un precedente applicativo che permette la connessione al server Openfire (XMPP) della piattaforma Meetecho e fornirà all'utente la possibilità di avviare la partecipazione alla conferenza gestendo attraverso un'interfaccia utente la possibilità di interrompere e sospendere l'ascolto della conferenza e di richiedere il permesso di poter inviare audio, cioè partecipare attivamente alla conferenza. L'utente non avrà accesso né ai dettagli di connessione al server SIP né alla scelta del numero da chiamare: l'applicativo sviluppato riceverà tali dettagli dal software precedentemente implementato.

2.1.2 Requisiti Operativi

Le funzioni svolte dall'applicativo da implementare riguardano l'interfacciamento con il server Asterisk e il demone di gestione del protocollo BFCP.

L'applicativo in particolare dovrà garantire:

- L'autenticazione al server Asterisk.
- L'avvio di una chiamata ad una conferenza.
- Un corretto *handshaking* dei *codec* da utilizzare in fase di comunicazione.
- L'invio e la ricezione dei flussi multimediali attraverso il protocollo RTP.
- La gestione del *re-invite* in caso di conferenza moderata da BFCP.
- L'invio di richieste di acquisizione/rilascio di risorsa moderata.
- La gestione delle richieste pervenute in modalità moderatore.
- La possibilità di mettere in pausa o passare alla modalità vivavoce la conferenza.

L'applicativo dovrà inoltre implementare i seguenti protocolli per garantire una comunicazione conforme agli standard internazionali:

- SIP (*Session Initiation Protocol*)
- RTP (*Real Time Protocol*)
- SDP (*Session Description Protocol*)
- BFCP (*Binary Floor Cotrol Protocol*)

Ecco di seguito un *Sequence Diagram* che riassume l'interazione tra l'utente e l'applicativo

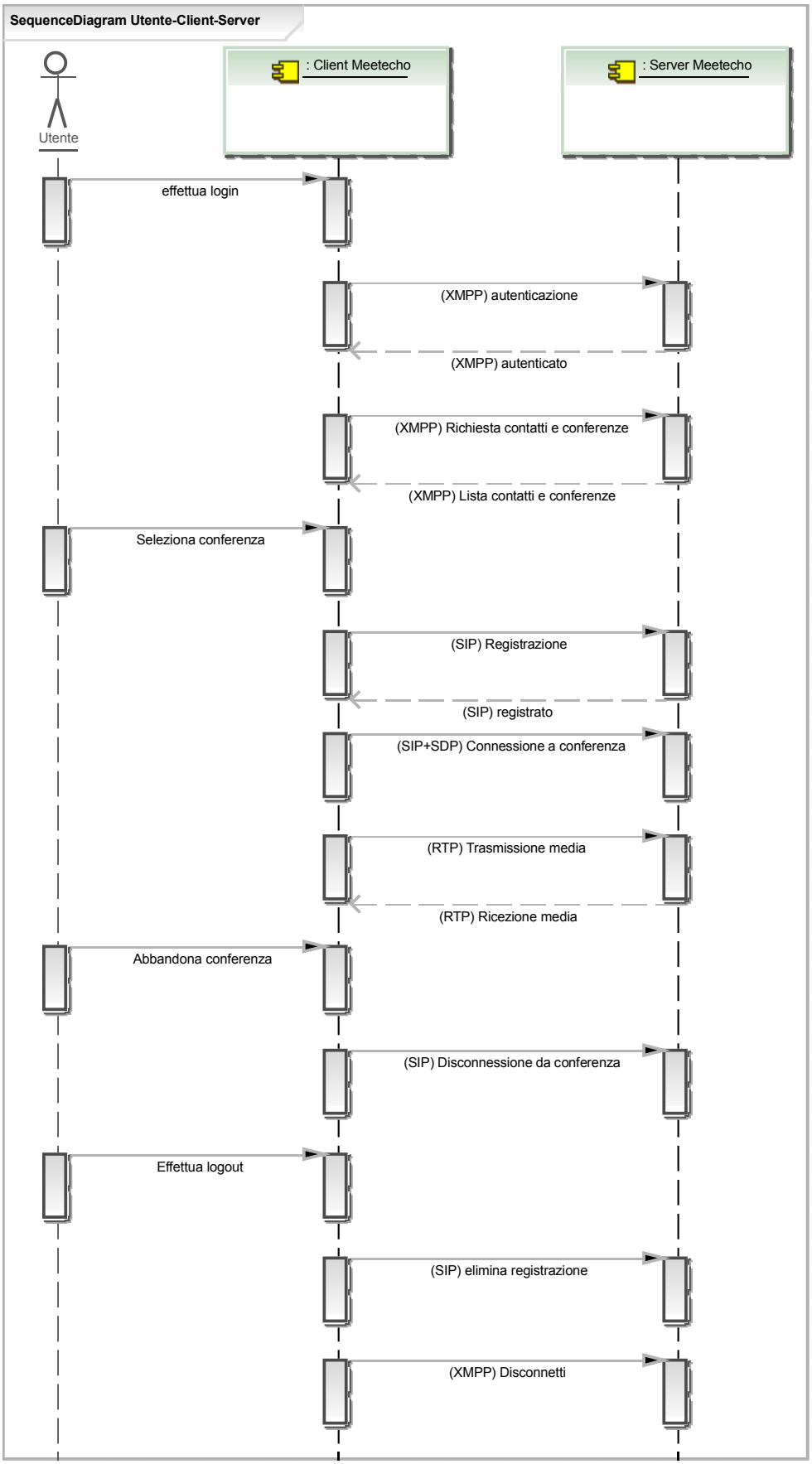


Figura 2.1: Sequence Diagram tra l'utente e l'applicativo

2.2 Progettazione dell'engine del client

Il client è stato progettato a partire dalla definizione di più blocchi funzionali interconnessi fra loro in base alle esigenze progettuali:

- Interfaccia grafica: per la diretta interazione con l'utente
- UserAgent: che si fa carico delle richieste ricevute dall'interfaccia utente e si collega ai rispettivi moduli (SIP-GestioneMedia-BFCP).
- Modulo SIP: per la gestione della REGISTER e degli INVITE al server Asterisk.
- Modulo gestione media: di supporto all'*handshaking* dei *codec* con il server SIP.
- Modulo Riproduzione/Registrazione Audio
- Modulo SDP *Parser*: per il *parsing* dei messaggi SDP ricevuti dal server SIP.
- Modulo RTP: per la gestione del protocollo di trasporto dei media.
- Modulo BFCP: per la gestione e la richiesta dei *Floor* al servizio di moderazione.
- Modulo CCMP(*Conference Control Manipulation Protocol*): per l'autenticazione come moderatore al server BFCP.

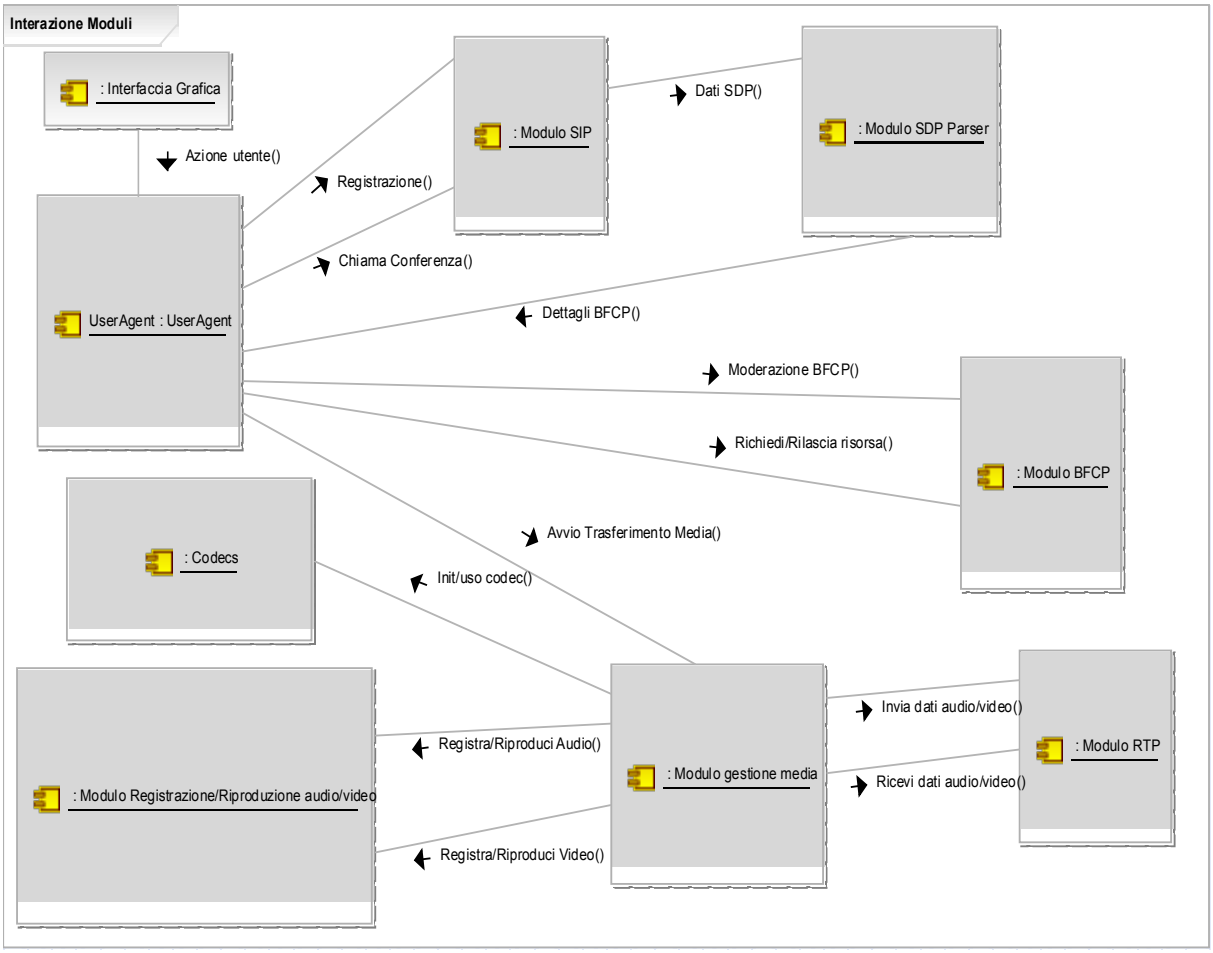


Figura 2.2: Interazione fra moduli

2.2.1 Interfaccia Grafica

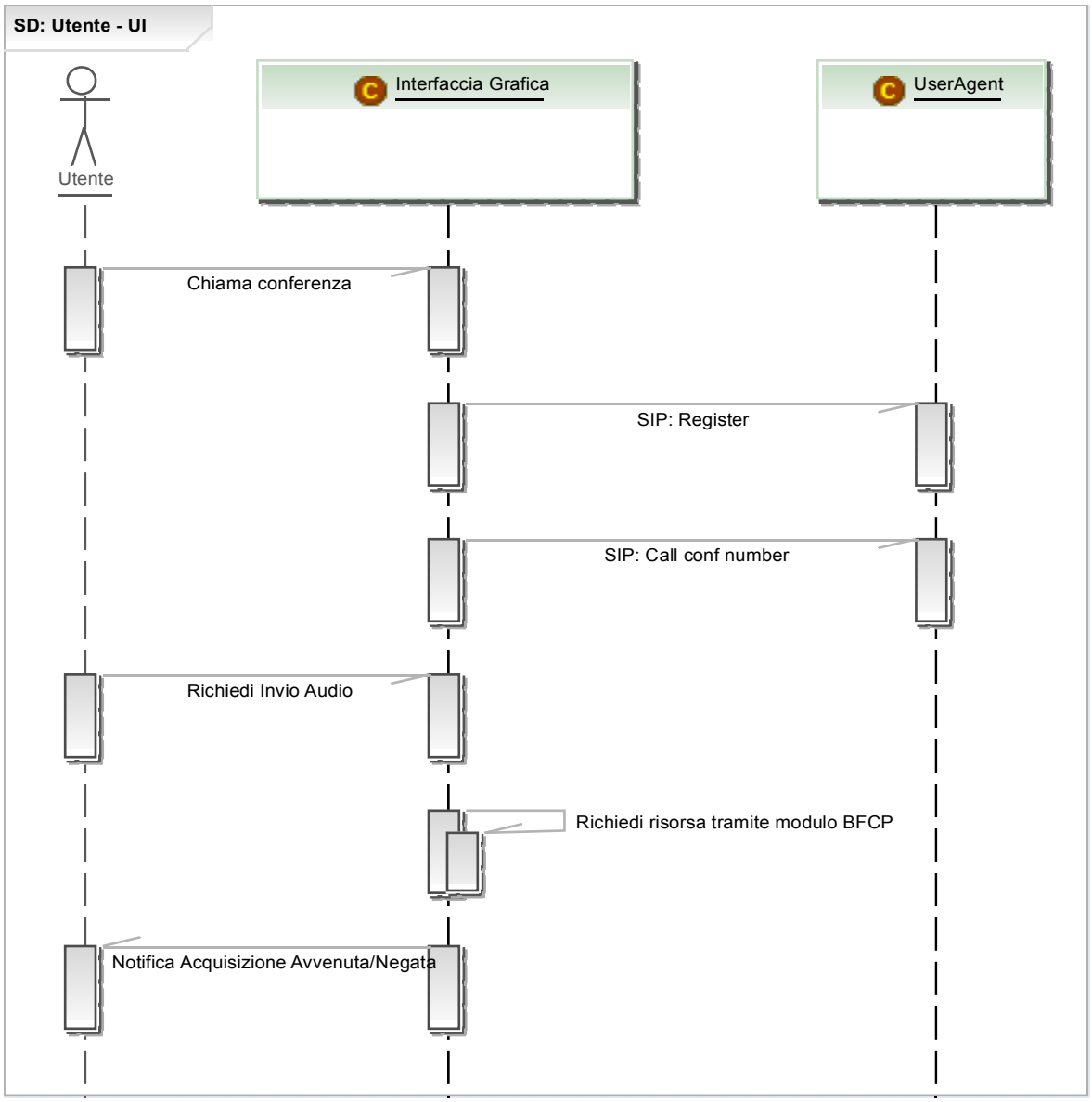


Figura 2.3: Sequence Diagram tra utente e interfaccia grafica

L'interfaccia grafica si occupa dell'interazione con l'utente, fornendo dei pulsanti in grado di permettere all'utente la connessione alla conferenza scelta lato XMPP.

L'utente, dopo essersi collegato alla conferenza con un apposito pulsante, inizierà ad ascoltare l'audio dei partecipanti attivi; l'utente appena collegato sarà invece un partecipante passivo e potrà solo ricevere audio senza inviarlo. Solo dopo aver ricevuto il re-INVITE del server contenente i dettagli necessari al collegamento al server BFCP, l'applicativo permetterà, attraverso un apposito pulsante, la richiesta di invio audio che

dovrà essere confermata e accettata/negata da un moderatore.

L'utente inoltre attraverso altri pulsanti avrà la possibilità di mettere in pausa o di terminare l'ascolto della conferenza in qualsiasi momento, oltre che alla possibilità di passare in modalità *vivavoce*.

Riepilogando le azioni che dovrà svolgere saranno:

- Iniziare l'ascolto di una conferenza attraverso un apposito pulsante.
- Possibilità del passaggio a modalità vivavoce o viceversa con apposito pulsante.
- Possibilità di mettere in pausa la conferenza.
- Terminare l'ascolto di una conferenza.
- Richiedere la possibilità di inviare audio (autorizzazione tramite BFCP).
- Possibilità di diventare moderatore della conferenza attraverso l'inserimento di una password.
- Gestire la moderazione della conferenza attraverso un apposito menu.

2.2.2 UserAgent

Il modulo UserAgent si occupa di gestire le richieste dell'utente ricevute attraverso l'interfaccia grafica. Tale modulo si metterà direttamente in contatto con il modulo SIP per la connessione al server e la partecipazione ad una conferenza, gestirà poi attraverso i dati ricevuti dal modulo SDP Parser la connessione al server BFCP attraverso il modulo BFCP.

Il modulo UserAgent dovrà inoltre dare la possibilità di richiedere risorse moderate attraverso il modulo BFCP e dovrà occuparsi della gestione del modulo CCMP (*Conference Control Manipulation Protocol*) permettendo all'utente di diventare moderatore della conferenza, ricevendo le relative richieste di assegnazione risorse da parte di utenti passivi.

Riepilogando le azioni che dovrà svolgere saranno:

- Collegamento e gestione del Modulo SIP.
- Collegamento e gestione del Modulo BFCP.
- Manipolazione dati ricevuti dal modulo SDP Parser.
- Connessione e invio richieste al server CCMP con relativo protocollo.

- Collegamento al modulo GestioneMedia per l'avvio dell'acquisizione/riproduzione dei contenuti multimediali.

2.2.3 Modulo SIP

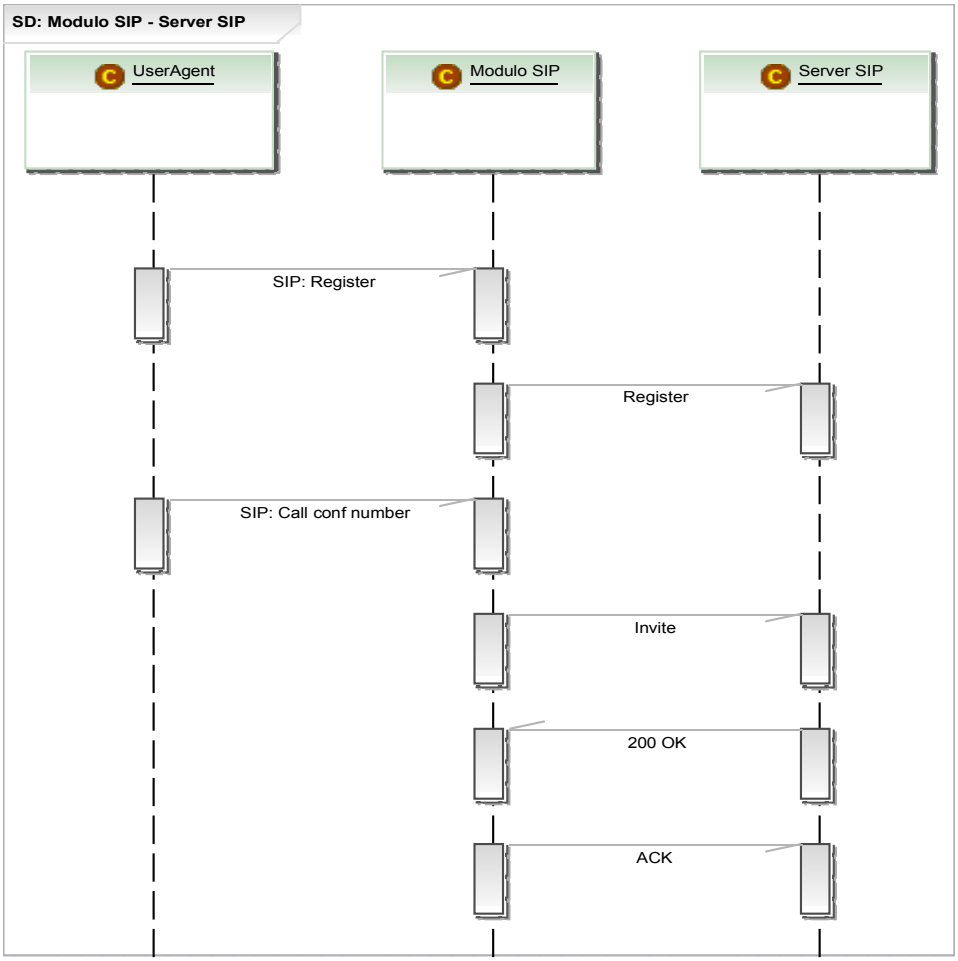


Figura 2.4: Sequence Diagram tra modulo SIP e server SIP

Il modulo SIP dovrà fornire gli strumenti necessari all'applicativo per gestire una sessione SIP. Il modulo dovrà quindi permettere ad un utente registrato sul server SIP di effettuare il login attraverso l'uso del messaggio SIP: *Register*. Successivamente dovrà permettere, quando richiesto, di procedere alla partecipazione ad una conferenza, gestendo correttamente il *threeway-handshaking* con messaggi *Invite* tipico del protocollo SIP, permettendo inoltre l'eventuale terminazione della chiamata SIP e il relativo *logout* dal server.

Riepilogando le azioni che dovrà svolgere saranno:

- Garantire un corretto collegamento al server SIP attraverso l'uso del relativo

protocollo con al gestione dei messaggi: Register, Bye, Cancel, Invite, Ack, Trying.

- Permettere la partecipazione ad una conferenza.
- Occuparsi della ricezione dei messaggi SDP e passarli al modulo SDP Parser per la relativa interpretazione.

2.2.4 Modulo gestione media

Il modulo gestione media si preoccupa di fornire una lista dei codec disponibili al modulo UserAgent, in modo che quest'ultimo permetta al modulo SIP di completare il *threeway-handshaking*. Il modulo gestione media permetterà inoltre il controllo dei *codec*, dove ogni *codec* avrà un interfaccia con dei metodi generici da poter chiamare all'occorrenza per la codifica, la decodifica. Ogni *codec* dovrà poi poter gestire l'eventuale modifica del *sample* rate e la possibilità di abilitazione/disabilitazione del suo utilizzo.

Il modulo gestione media si occupa inoltre del collegamento al modulo riproduzione/registrazione ricevendo i buffer audio/video da inviare al Modulo RTP opportunamente codificati e viceversa.

Riepilogando le azioni che dovrà svolgere saranno:

- Gestione dei *codec*.
- Opportuno collegamento con il modulo gestione media in modo da garantire prelievo di buffer da codificare e invio di buffer da riprodurre.
- Comunicazione con il modulo UserAgent in modo da fornire un elenco dei *codec* disponibili al modulo SIP necessari per completare il *threeway-handshaking*.

2.2.5 Modulo Riproduzione/Registrazione Audio

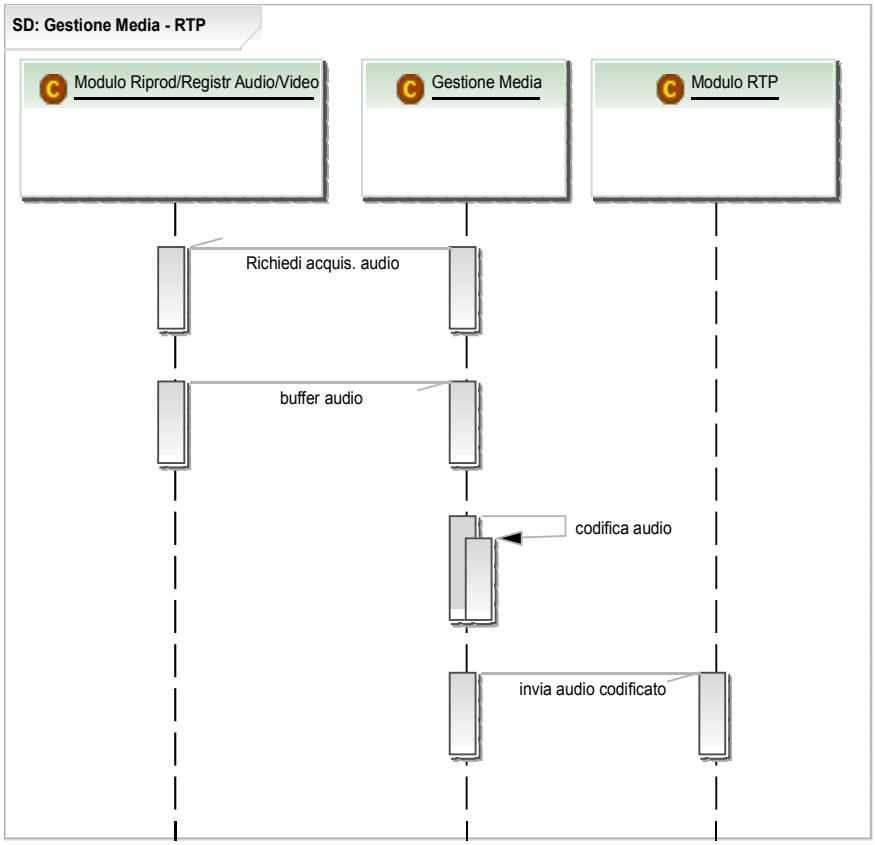


Figura 2.5: Sequence Diagram tra modulo Gestione Media e RTP

Tale modulo si occupa della creazione/gestione dei buffer audio ottenuti dal dispositivo di acquisizione oppure ricevuti dal modulo RTP. Questo modulo si preoccuperà quindi di riprodurre l'audio ricevuto da rete e di registrare l'audio dal microfono del dispositivo. L'audio acquisito/ricevuto prima di essere passato al modulo RTP dovrà essere opportunamente codificato/decodificato attraverso il *codec* scelto in fase di *signaling*, attraverso il *threeway-handshaking* con il server SIP.

Riepilogando le azioni che dovrà svolgere saranno:

- Acquisizione dai dispositivi multimediali (Microfono, Videocamera)
- Riproduzione di flussi multimediali in ingresso (Audio/Video)

2.2.6 Modulo SDP Parser

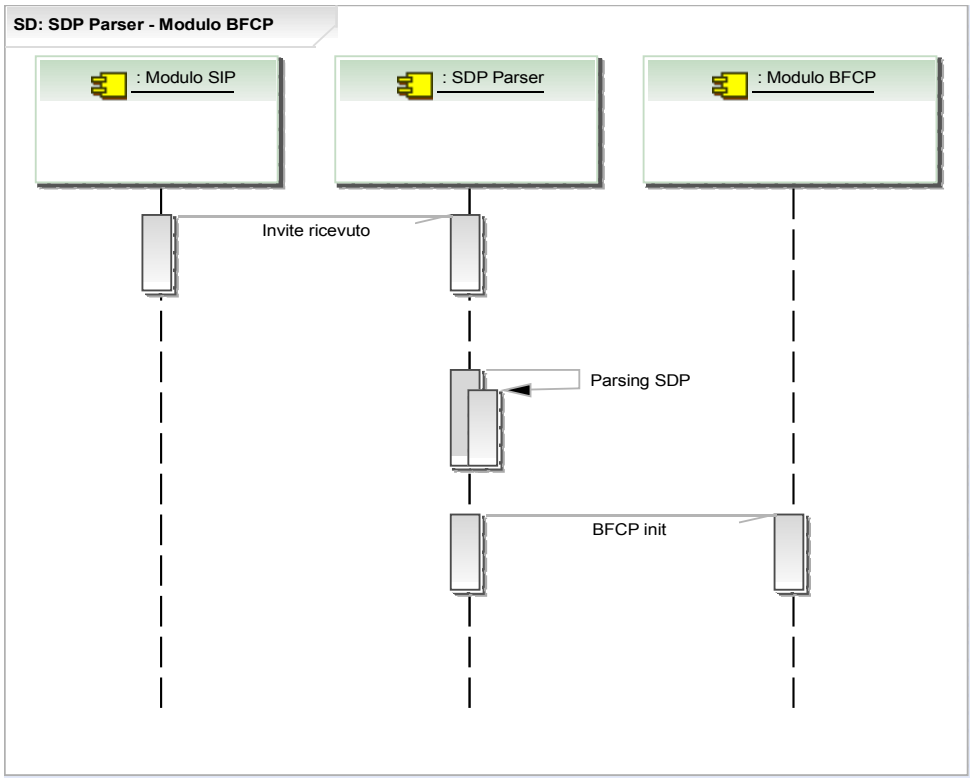


Figura 2.6: Sequence Diagram tra modulo SDP Parser e BFCP

Il modulo SDP *Parser* si preoccupa di interpretare i messaggi SDP ricevuti nelle transazioni del protocollo SIP per intercettare i *codec* utilizzati dal server, interpretando i *mediafield*, e fornire agli altri blocchi queste informazioni per permettere la completa negoziazione dei *codec* da utilizzare durante il collegamento alla conferenza.

Il modulo SDP Parser inoltre dovrà intercettare i parametri relativi al collegamento al server BFCP in caso di conferenza moderata. Il modulo alla ricezione di un *re-Invite* su una chiamata attiva effettua il *parsing* dei parametri BFCP nel messaggio SDP, tali parametri vengono poi passati alle funzioni di inizializzazione del modulo BFCP.

Riepilogando le azioni che dovrà svolgere saranno:

- Effettuare il *parsing* dei *MediaFields* e dei *MediaDescriptors* necessari per la corretta decisione dei *codec* da comunicare durante il *threeway-handshaking*.
- Effettuare il *parsing* e la memorizzazione dei dettagli necessari per la connessione al server di moderazione BFCP.

2.2.7 Modulo RTP

Il modulo RTP si occupa del trasporto vero e proprio dei dati multimediali su rete, i dati vengono incapsulati in pacchetti RTP e scambiati con il server Asterisk. Il modulo si preoccupa di verificare inoltre, alla ricezione, l'ordine dei pacchetti ricevuti attraverso il *TimeStamp* e il *SequenceNumber* indicativo in ogni pacchetto, successivamente passa i dati ricevuti al modulo riproduzione/registrazione audio. I pacchetti sono contraddistinti dal campo *PayloadType* che ne identifica il contenuto multimediale stabilito in fase di *signaling*. Il modulo dovrà implementare il protocollo RTP sulla base di UDP, definito nella rfc3550 dello IETF.

Riepilogando le azioni che dovrà svolgere saranno:

- Pacchettizzare/De-pacchettizzare secondo le specifiche del protocollo RTP i *buffer* audio/video inviati/ricevuti.
- Riordinare e disporre in sequenza i pacchetti ricevuti, dato che RTP nell'ambiente considerato sarà utilizzato sulla base di UDP, protocollo che non garantisce l'ordine dei pacchetti ricevuti.

2.2.8 Modulo BFCP

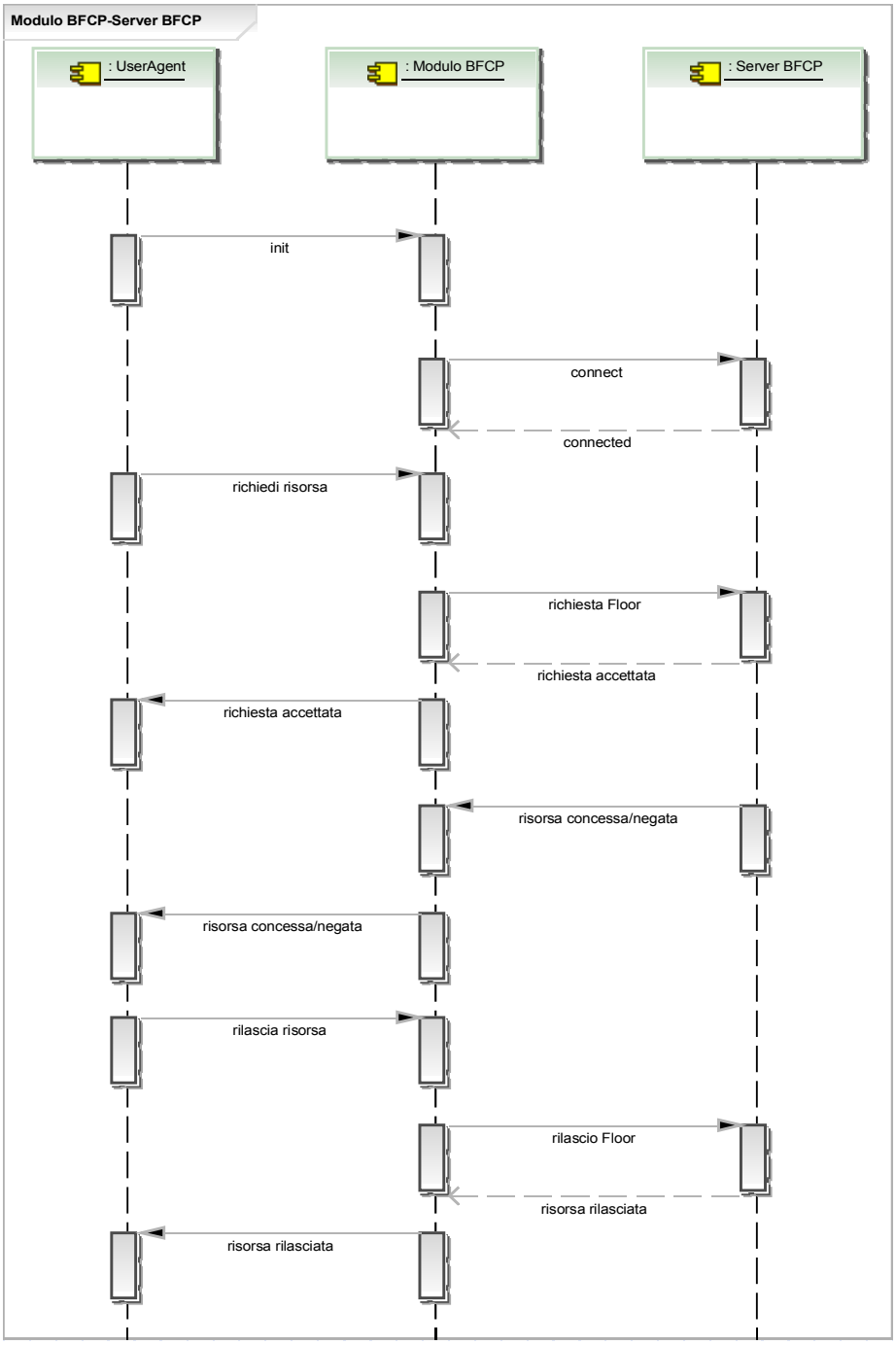


Figura 2.7: Sequence Diagram tra modulo BFCP e server BFCP

Il modulo BFCP è utilizzato laddove il *client* si collega ad una conferenza moderata, in tal caso il modulo viene inizializzato con i parametri intercettati dal modulo SDP *Parser*.

Il modulo si occuperà delle richieste relative alle risorse (*floor*) disponibili e della gestione delle risposte inviate dal server. Il modulo dovrà inoltre gestire la modalità *moderatore (chair)* dove l'utente, attraverso un'apposita interfaccia grafica, potrà approvare o negare le

richieste di acquisizione risorsa relative ad utenti passivi. Tale modulo dovrà quindi collegarsi al server utilizzando il protocollo binario BFCP sulla base del protocollo di trasporto TCP.

Riepilogando le azioni che dovrà svolgere saranno:

- Connessione al server BFCP attraverso i parametri ricevuti in fase di inizializzazione.
- Gestire indipendentemente eventuali richieste/rilasci di *Floor* (risorse) ricevute dal modulo UserAgent.
- Garantire al modulo UserAgent la moderazione della conferenza, fornendo i metodi necessari all'approvazione di richieste provenienti da utenti passivi della conferenza che richiedono in questo modo la partecipazione attiva alla stessa.

2.2.9 Modulo CCMP

Il modulo CCMP potrà essere utilizzato solo dopo che il modulo BFCP avrà stabilito una connessione con il server BFCP della piattaforma Meetecho.

Tale modulo dovrà quindi garantire la richiesta e il rilascio al server CCMP di una risorsa da moderare con l'uso di un'opportuna chiave numerica segreta associata alla particolare conferenza a cui il modulo SIP si è collegato. Il modulo dovrà inoltre prevedere la possibilità di invio chiave errata e relativa notifica al modulo chiamante.

Capitolo 3
Implementazione

Come già esposto nei paragrafi precedenti l'ambiente di sviluppo Eclipse con *plugin* per Android è basato sul linguaggio Java. In questo capitolo verranno descritte in maniera esaustiva tutte le scelte implementative adottate nel corso della realizzazione del nostro *client*, mostrando tutte le soluzioni utilizzate per lo sviluppo dell'applicazione. Quindi avremo modo di approfondire e ad analizzare le problematiche affrontate sia per lo sviluppo della parte relativa alla logica implementativa SIP (S*ession Initiation Protocol*), RTP (*RealTime Transport Protocol*) e BFCP (*Binary Floor Control Protocol*), nonché della parte relativa all'interfaccia utente.

3.1 Scelte implementative

Nella fase iniziale dell'implementazione si sono valutate e testate le funzionalità di librerie già esistenti valutando la possibilità di adottarle nell'implementazione del *client* in questione.

Si è deciso di adottare la libreria MjSip per la gestione delle connessioni e relative richieste al server SIP poiché tale libreria era stata già utilizzata in precedenti lavori di tesi su *client*, piattaforme e ambienti differenti, dimostrando stabilità e funzionalità richieste.

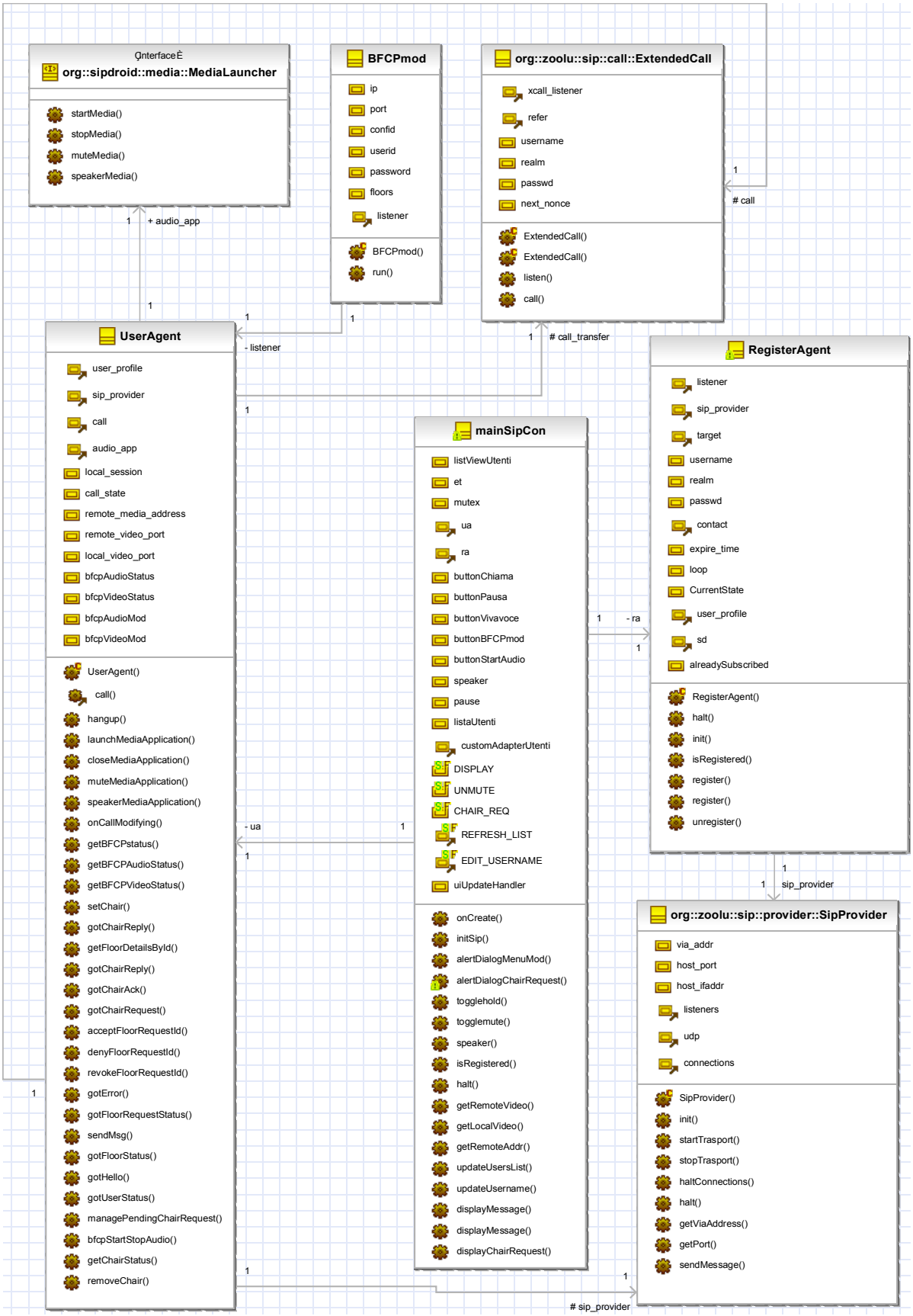
Per l'implementazione del trasporto tramite protocollo RTP e della codifica dei flussi audio si è deciso di utilizzare alcune librerie relative al progetto o*pen source*: SipDroid, tale *client* è stato provato in fase di testing delle alternative esistenti.

Per l'implementazione del protocollo BFCP è stata utilizzata una libreria frutto di un lavoro precedente di tesi, tale libreria è stata adattata alle esigenze specifiche del progetto, effettuando modifiche parziali, cercando ove possibile di ridurle al minimo.

Per l'implementazione di tutti gli altri componenti si è partiti da zero considerando le funzionalità da implementare e le eventuali limitazioni dovute all'ambiente Android.

3.2 Diagramma delle classi implementate

Durante l'analisi dell'implementazione è opportuno far riferimento al seguente diagramma delle classi UML che fornisce le associazioni tra i moduli implementati:



3.3 Implementazione modulo SIP

Come è stato accennato precedentemente per l'implementazione del modulo SIP è stata utilizzata la libreria MjSip sviluppata dall'Università di Parma e successivamente adattata all'ambiente Android dal team di sviluppo del progetto *OpenSource*: SipDroid.

Tale libreria permette la corretta registrazione ad un server SIP e lo scambio dei relativi messaggi, nel caso in esame permette la connessione al server Asterisk della piattaforma Meetecho. È composta dai seguenti componenti (*packages*) necessari alla completa comunicazione con il server SIP:

- `Address`: si occupa della gestione dei *SipURL* necessari alla comunicazione *client-server*.
- `Authentication`: si occupa dell'autenticazione presso il server SIP.
- `Call`: si occupa della creazione degli appositi messaggi di *Invite* per la richiesta di comunicazione presso il server.
- `Dialog`: si occupa di tenere traccia delle conversazioni tra *client-server*.
- `Header`: gestisce i vari tipi di *header* utilizzabili nel protocollo SIP.
- `Message`: si occupa della creazione e gestione dei messaggi SIP.
- `Provider`: si occupa di tutto il processo di comunicazione con il server SIP, utilizza gli altri componenti per stabilire un collegamento e scambiare messaggi con il server SIP.
- `Transaction`: si occupa di gestire la corrispondenza tra richieste e risposte scambiate nella comunicazione tra *client-server*.

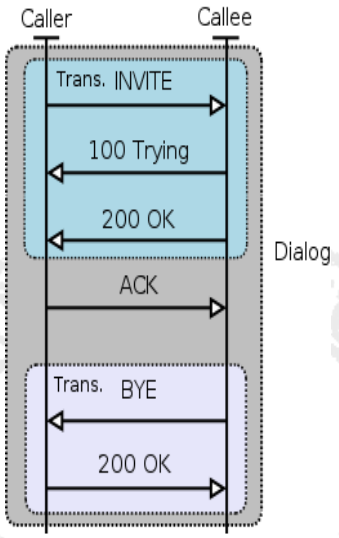
Fondamentale importanza tra questi componenti assume il *package* Provider, contenente al suo interno la classe `SipProvider`. Tale classe permette allo sviluppatore di istanziare un oggetto di tipo `SipProvider` con indirizzo ip e porto del server SIP permettendo la connessione e lo scambio di messaggi con il server.

```
/** Creates a new SipProvider. */
        public SipProvider(String via_addr, int port) {
                init(via_addr, port, null, null);
                initlog();
                startTrasport();
        }
```

L'oggetto di tipo `SipProvider` sarà utilizzato in tutte le operazioni di comunicazione con il server SIP e verrà utilizzato per la creazione dell'oggetto di tipo `UserAgent` che utilizzerà l'oggetto `SipProvider` per tutto il periodo di comunicazione tra *client-server*. I metodi di notevole importanza per tale oggetto e che consentiranno un suo corretto uso sono:

- `sendMessage(Message msg)`: che viene utilizzato per inviare un messaggio opportunamente incapsulato nella struttura dati `Message`.
- `halt()`: che viene utilizzato per interrompere la comunicazione con il server SIP.

Fondamentale importanza nella libreria MjSip assume il *package* `Call`, in particolar modo la classe `Call` utilizzata dall'oggetto di tipo `UserAgent` che permette all'applicativo l'avvio di una comunicazione verso un particolare contatto o numero presente sul server SIP. In merito all'uso di tale classe è utile far riferimento al seguente schema che mostra la struttura tipica di una sessione SIP:



All'atto della creazione dell'oggetto di tipo `Call` bisogna fornire al costruttore della classe un oggetto di tipo `SipProvider`, prima descritto, che permetterà la comunicazione con il server SIP, come mostrato di seguito:

```
Call call = new Call(sip_provider, from_url,user_profile.contact_url, this);
```

In particolare il costruttore della classe `Call` memorizza tali dettagli per utilizzarli durante la gestione della comunicazione.

```
public class Call implements InviteDialogListener {
[...]
	/** Creates a new Call. */
	public Call(SipProvider sip_provider, String from_url, String contact_url,
CallListener call_listener) {
		this.sip_provider = sip_provider;
		this.log = sip_provider.getLog();
		this.listener = call_listener;
		this.from_url = from_url;
		this.contact_url = contact_url;
		this.dialog = null;
		this.local_sdp = null;
		this.remote_sdp = null;
	}
```

L'oggetto di tipo `Call` implementando la classe `InviteDialogListener` riceverà le risposte relative ai messaggi di tipo *Invite* necessari al completamento del three-way-*handshaking* per la contrattazione dei *codec* per i media utilizzati nella comunicazione. La gestione della comunicazione in tale classe è effettuata attraverso l'uso opportuno di oggetti di tipo `Dialog`, contenitori dei messaggi scambiati tra *client-server.*

I metodi che svolgono un ruolo di notevole importanza nel processo di comunicazione di tale classe sono:

- `call(...)`: che permette l'effettivo avvio della comunicazione verso la destinazione prescelta, opportunamente registrata sul server.

```
/** Starts a new call, inviting a remote user (callee) */
	public void call(String callee, String from, String contact, String sdp) {
		printLog("calling " + callee, LogLevel.HIGH);
		if (from == null)
			from = from_url;
		if (contact == null)
			contact = contact_url;
		if (sdp != null)
			local_sdp = sdp;
		dialog = new InviteDialog(sip_provider, this);
		if (local_sdp != null)
			dialog.invite(callee, from, contact, local_sdp);
		else
			dialog.inviteWithoutOffer(callee, from, contact);
	}
```

- `hangup()`: che permette la terminazione di una comunicazione attiva.

```
/**
 * Closes an ongoing or incoming/outgoing call
 * <p>
 * It trys to fires refuse(), cancel(), and bye() methods
 */
public void hangup() {
        if (dialog != null) { // try dialog.refuse(),cancel(), and bye()
                dialog.refuse();
                dialog.cancel();
                dialog.bye();
        }
}
```

3.4 Implementazione Modulo SDP Parser

L'implementazione del modulo SDP *Parser* è iniziata a partire dalla classe `SdpParser` della libreria MjSip, tale classe permette allo sviluppatore di istanziare un oggetto capace di effettuare il *parsing* di un messaggio SDP estrapolato dal modulo SIP nei messaggi scambiati con il server. Il modulo in particolar modo permette il *parsing* di `SdpField`, `SdpMediaField` e `SdpMediaDescriptor`, necessari al modulo UserAgent per la gestione della contrattazione dei *codec* necessari alla comunicazione multimediale. La classe inoltre è stato opportunamente modificata per permettere al modulo UserAgent di reperire i dettagli necessari a stabilire un corretto collegamento con il server BFCP della piattaforma Meetecho. I moduli di tale classe sono:

- `parseMediaField()`: effettua il *parsing* di un MediaField, cioè di un campo media del protocollo SDP.
- `parseMediaDescriptor()`: effettua il *parsing* di un MediaDescriptor, cioè di un descrittore di uno stream multimediale.
- `ParseReinviteAttributes()`: è una delle funzioni inserite nella classe `SdpParser`, è utilizzata per effettuare il *parsing* degli attributi necessari alla connessione al server BFCP ricevuti in un *re-Invite* da parte del server SIP della piattaforma Meetecho.
- `GetBindings()`: è un'altra funzione inserita nella classe che fornisce un `HashMap` contenente tutte le informazioni, contenute nel pacchetto SDP allegato al messaggio di *reInvite* ottenuto dal server SIP, che potrebbe essere necessarie ad un eventuale *UserAgent* che debba gestire il collegamento al server BFCP.

Andando ad analizzare singolarmente le due funzioni aggiunte notiamo che nella funzione `ParseReinviteAttributes()` il *parsing* degli attributi viene effettuato sfruttando le funzioni della classe contenitore `MediaDescriptor` inclusa nel *package* SDP della libreria MjSip.

L'oggetto di tipo `MediaDescriptor` viene creato sfruttando il metodo `parseMediaDescriptor()`, a partire da tale oggetto viene controllato il *tag* "application" del *MediaDescriptor* per verificare se il contenuto corrisponde al protocollo di trasporto BFCP, in caso positivo viene salvato il porto destinazione del server BFCP, successivamente si passa al *parsing* dei *label* audio e video, in caso vengano trovati permetteranno al metodo `ParseReinviteAttributes()` di effettuare il *parsing* anche dei *floorid* per l'audio e il video, necessari al modulo UserAgent per garantire una corretta richiesta delle risorse moderate dal server BFCP all'atto dell'interazione dell'utente con l'interfaccia grafica.

```
public void parseReinviteAttributes() {
        MediaDescriptor md;
        while (index < str.length()) {
                md = parseMediaDescriptor();
                if (md.m.getMedia().equalsIgnoreCase("application")
                                        && md.m.getTransport().contains("BFCP")) {
                String port = ((Integer) md.m.getPort()).toString();
                [...]
                v = md.getAttributes("floorid");
                if (bindings.get("audioLabel") != null) {
                for (int i = 0; i < v.size(); i++) {
                        if (((AttributeField) v.get(i)).getValue().contains(
                                "m-stream:" + bindings.get("audioLabel"))) {
                bindings.put("audioFloorid", ((AttributeField)
                                v.get(i)).getValue().split("floorid:")[1].split(" ")[0]);
                } }
                }
                if (bindings.get("videoLabel") != null) {
                for (int i = 0; i < v.size(); i++) {
                        if (((AttributeField) v.get(i)).getValue().contains("m-stream:" +
                                bindings.get("videoLabel"))) {
                        bindings.put("videoFloorid", ((AttributeField)
                                v.get(i)).getValue().split("floorid:")[1].split(" ")[0]);
                        }

                } }
                } else if (md.m.getMedia().equalsIgnoreCase("audio")) {
                        if (md.getAttribute("label") != null) {
                        bindings.put("audioLabel", md.getAttribute("label").value
                                                .split("label:")[1]);
                        }
                        } else if (md.m.getMedia().equalsIgnoreCase("video")) {
                        if (md.getAttribute("label") != null) {
                        bindings.put("videoLabel", md.getAttribute("label").value
                                                .split("label:")[1]);
                        } }
        } }
```

La funzione `getBindings()` viene utilizzata dal modulo UserAgent per acquisire l'oggetto di tipo `Map` contenente le informazioni ricavate effettuando il *parsing* del pacchetto SDP contenuto nel messaggio SIP.

```
public Map<String, String> getBindings() {
        return bindings;
}
```

3.5 Implementazione modulo RTP

Per l'implementazione del modulo RTP è stato utilizzato un *package* disponibile nel progetto OpenSource: SipDroid, composto principalmente da tre classi:

- `RtpSocket`: La classe ha il compito di decapsulare la struttura dati di un pacchetto RTP precedentemente creato, incapsularlo in un pacchetto UDP (*User Datagram Protocol*) e inviarlo tramite la *socket* di tipo SipdroidSocket o viceversa.
- `RtpPacket`: La classe ha il compito di incapsulare/decapsulare i dati da inviare/ricevere attraverso il protocollo RTP.
- `SipdroidSocket`: Tale classe estende `DatagramSocket` e si occupa realmente del processo di invio e ricezione di pacchetti tramite protocollo di trasmissione UDP.

Andando ad analizzare in dettaglio la classe `RtpSocket` si può notare come al suo interno siano conservate istanze di una *socket* di tipo `SipdroidSocket` e un pacchetto di tipo `DatagramPacket`. All'atto della ricezione di un pacchetto RTP i dati ricevuti attraverso la *socket* UDP vengono scritto nel buffer passato attraverso il pacchetto RTP fornito come argomento al metodo `receive(RtpPacket rtpp)`.

All'atto dell'invio di un pacchetto invece, il pacchetto RTP ricevuto come argomento del metodo `send(RtpPacket rtpp)` viene utilizzato per creare un pacchetto UDP da inviare attraverso la `DatagramSocket`.

```
public class RtpSocket {
        SipdroidSocket socket;
        DatagramPacket datagram;
        InetAddress r_addr;

        /** Remote port */
        int r_port;
        [...]
        /** Receives a RTP packet from this socket */
        public void receive(RtpPacket rtpp) throws IOException {
                datagram.setData(rtpp.packet);
                datagram.setLength(rtpp.packet.length);
                socket.receive(datagram);
                if (!socket.isConnected())
                        socket.connect(datagram.getAddress(),datagram.getPort());
                rtpp.packet_len = datagram.getLength();
        }

        /** Sends a RTP packet from this socket */
        public void send(RtpPacket rtpp) throws IOException {
                datagram.setData(rtpp.packet);
                datagram.setLength(rtpp.packet_len);
                datagram.setAddress(r_addr);
                datagram.setPort(r_port);
                socket.send(datagram);
        }
[...]
}
```

3.6 Implementazione GestioneMedia e Registrazione/Riproduzione Audio

Nella fase progettuale si era pensato di rendere indipendenti i moduli di GestioneMedia e Registrazione/Riproduzione Audio ma si è deciso poi in fase implementativa di utilizzare classi uniche che implementassero tali funzioni. Il *package* utilizzato per tale gestione è stato riadattato a partire da un *package* del progetto *OpenSource*: SipDroid. Tale *package* presentava dipendenze specifiche dal progetto originario ed è stato adattato per gli scopi di questo lavoro di tesi.

Prima di analizzare il *package* è necessario far riferimento al *package* utilizzato per la gestione dei *codec* disponibili in fase di *three-way-handshaking* effettuato dal modulo SIP durante una *call* e utilizzabili dal modulo GestioneMedia per codifica/decodifica dei flussi audio.

Tale *package* si presenta composto da varie classi contenenti l'implementazione di uno specifico *codec* a partire da una stessa interfaccia comune:

```
public interface Codec {
        int decode(byte encoded[], short lin[], int size);
        int encode(short lin[], int offset, byte alaw[], int frames);
        int samp_rate();
        int frame_size();
        void init();
        [...]
}
```

I *codec* implementati e utilizzati dai moduli dell'applicativo sono: U-LAW, A-LAW e GSM. Il *package* GestioneMedia in esame è composto da più classi ognuna con delle funzionalità specifiche:

- `MediaLauncher`: Interfaccia che definisce i metodi generici per la gestione di un contenuto multimediale.
- `JAudioLauncher`: Classe che implementa l'interfaccia `MediaLauncher` e che si occupa della gestione di contenuti multimediali audio attraverso l'uso combinato di oggetti di tipo `RtpStreamReceiver/RtpStreamSender`.
- `RtpStreamReceiver`: Classe che si occupa della ricezione di pacchetti RTP opportunamente decapsulati e della successiva decodifica attraverso il *codec* selezionato nel modulo SIP al fine di avviare la riproduzione del contenuto multimediale audio attraverso le API dell'ambiente Android.
- `RtpStreamSender`: Classe che si occupa dell'acquisizione del flusso audio da una periferica di ingresso (microfono) e della successiva codifica attraverso il *codec* selezionato nel modulo SIP al fine di inviare tale flusso codificato opportunamente incapsulato al modulo RTP sottostante.

Entrambi le classi `RtpStreamSender` e `RtpStreamReceiver` estendono la classe `Thread` e dopo una fase di inizializzazione vengono avviate attraverso il metodo `run()` in cui svolgono tutte le principali azioni che gli competono.

In particolare la classe `RtpStreamReceiver` si occupa della ricezione del flusso multimediale attraverso la struttura dati `rtp_packet` che viene creata attraverso la

classe `RtpPacket` dando come argomento un *buffer* di byte nel quale verra decapsulato il flusso audio contenuto nel pacchetto RTP. Tale *buffer* di byte sarà decodificato attraverso i metodi generici del *codec* concordato nel modulo SIP e sarà poi scritto nel buffer del dispositivo hardware per essere riprodotto attraverso il metodo `write()` della classe nativa di Android: `AudioTrack`.

```
public class RtpStreamReceiver extends Thread {
[...]
RtpSocket rtp_socket = null;
RtpPacket rtp_packet;
AudioTrack track;
[...]
public void run() {
        [...]
        byte[] buffer = new byte[BUFFER_SIZE + 12];
        rtp_packet = new RtpPacket(buffer, 0);
        running = true;
        short lin[] = new short[BUFFER_SIZE];
        int len=0;
        [...]
        setCodec();
        [...]
        track.play();
        while (running) {
        [...]
        try {
        rtp_socket.receive(rtp_packet);
        [...]
                len = p_type.codec.decode(buffer, lin, rtp_packet.getPayloadLength());
                [...]
                user += track.write(lin, 0, len);
        [...]
        } catch (IOException e) {
                [...]
        }
        [...]
}
[...]
}
```

La classe `RtpStreamSender` invece si occupa dell'acquisizione del flusso audio da una periferica di ingresso (microfono) attraverso le API dell'ambiente Android con l'uso della classe `AudioRecord`. Innanzitutto viene creato un buffer con il quale si inizializza il pacchetto RTP, il buffer sarà riempito con il flusso dati, opportunamente codificato dal *codec* prescelto in fase di negoziazione dal modulo SIP, dopo essere stato ottenuto dal metodo `read()` della classe nativa `AudioRecord` di Android. Dopo il processo di acquisizione e codifica il pacchetto sarà inviato attraverso la *socket* RTP.

```
public class RtpStreamSender extends Thread {
[...]
public void run() {
        byte[] buffer = new byte[frame_size + 12];
        RtpPacket rtp_packet = new RtpPacket(buffer, 0);
        rtp_packet.setPayloadType(p_type.number);
        [...]
        AudioRecord record = new AudioRecord(MediaRecorder.AudioSource.MIC,
p_type.codec.samp_rate(), AudioFormat.CHANNEL_CONFIGURATION_MONO,
AudioFormat.ENCODING_PCM_16BIT, min);
        short[] lin = new short[frame_size*11];
        [...]
        p_type.codec.init();
        record.startRecording();
        while (running) {
                [...]
                num = record.read(lin,(ring+delay*frame_size)%(frame_size*11),frame_size);
                [...]
                num = p_type.codec.encode(lin, ring%(frame_size*11), buffer, num);
ring += frame_size;
                rtp_packet.setSequenceNumber(seqn++);
                rtp_packet.setTimestamp(time);
                rtp_packet.setPayloadLength(num);
                try {
                        rtp_socket.send(rtp_packet);
                } catch (IOException e) {
        }
        [...]
}
[...]
}
```

Istanze di tali classi saranno utilizzate nella classe `JAudioLauncher` per gestire simultaneamente l'invio e la ricezione di flussi multimediali:

```
public class JAudioLauncher implements MediaLauncher
{  [...]
   public boolean startMedia()
   {  if (sender!=null)     {  sender.start(); }
      if (receiver!=null)   {  receiver.start(); }
      return true;
   } [...]
}
```

3.7 Implementazione modulo BFCP

Il modulo BFCP è stato implementato utilizzando una libreria frutto di un lavoro di tesi precedente. Tale libreria è stata adattata alle esigenze specifiche del progetto, effettuando modifiche parziali ad alcune classi. La libreria è organizzata in un *package* composto da una classe di controllo e un *dispatcher* per i *listener* che desiderano essere informati su eventuali risposte a richieste effettuate al server BFCP attraverso la classe di controllo o richieste relative a risorse in moderazione dal client.

Al costruttore della classe di controllo `BFCPControlCenter` vengono passati come argomento l'*userid* dell'utente connesso al server SIP e la conferenza a cui si è collegati:

```
public BFCPControlCenter(int userID1,long confID1)
```

Successivamente sarà possibile utilizzare sull'oggetto istanziato il metodo `connect(...)` per effettuare la connessione al server BFCP attraverso le informazioni reperite dal modulo SDP Parser. A questo punto il modulo UserAgent avrà la possibilità di inoltrare la richiesta di una risorsa moderata al moderatore della conferenza o rilasciare una risorsa precedentemente ottenuta attraverso i seguenti metodi:

```
public void helloServer()

public void requestFloor(int floorID)

public int releaseFloor(int floorRequestID)
```

Il metodo `helloServer()` fa partire la comunicazione tra client-server.

Il metodo `requestFloor(...)` invece da la possibilità ad un *participant* della conferenza di richiedere una risorsa moderata avente un determinato *floorID*.

Il metodo `releaseFloor(...)` invece da la possibilità di rilasciare una risorsa precedentemente acquisita passando come argomento il *FloorRequestID* associato alla relativa richiesta di acquisizione.

Dopo la chiamata del metodo `connect(...)` e dopo essersi opportunamente autenticati con il modulo CCMP sarà possibile inoltre dichiararsi *Chair* (Moderatore) al server BFCP,

ricevendo così notifiche di eventuali richieste di acquisizione risorse per cui si è ottenuto la moderazione attraverso il modulo CCMP:

```
public Chair beTheChair(boolean chair)
```

Tale funzione restituirà un oggetto di tipo `Chair` creato per l'invio di messaggi di moderazione al server BFCP, necessari per approvare, negare o revocare una risorsa:

```
public int acceptFloorRequest(int floorRequestID)

public int denyFloorRequest(int floorRequestID)

public int revokeFloorRequest(int floorRequestID)

public void revokeFloorRequest(int floorRequestID, int[] floors)
```

Tutti i metodi di moderazione precedenti utilizzano come argomento il `floorRequestID` un intero rappresentante univocamente presso il server BFCP la richiesta effettuata dall'utente, a partire da tale numero identificativo infatti sarà possibile risalire ad ulteriori informazioni quali ad esempio l'ID dell'utente, la risorsa richiesta e la conferenza di riferimento. In particolare è stata introdotta la funzione `revokeFloorRequest(int floorRequestID, int[] floors)`, in quanto il metodo originario ritorna valore "`-1`" nel caso in cui il *floorRequestID* non è presente nella lista della richieste ricevute, tuttavia un utente potrebbe aver ottenuto una risorsa prima che il moderatore abbia effettuato l'accesso, pertanto è necessario che l'istanza della classe `UserAgent` invochi il metodo alternativo fornendo come argomento aggiuntivo la lista dei *floor* da revocare.

La classe *dispatcher* `BFCPDispatcher` è una classe statica che da la possibilità alle istanze di classi che implementano l'interfaccia pubblica `BFCPListener` di aggiungersi come *listener* ed essere quindi informati di eventuali risposte e/o notifiche ricevute dal server BFCP.

Quindi è utile mostrare la struttura di tale interfaccia per esaminare le funzionalità implementate da ogni singolo metodo:

```
public interface BFCPListener {
        public void gotHello(long conferenceID,int userID);

        public void gotFloorRequestStatus(long conferenceID,int userID,BFCPFloorRequestInfo
details);

        public void gotUserStatus(long conferenceID,int userID,String user);

        public void gotFloorStatus(long conferenceID,int userID,int floor);

        public void gotError(long conferenceID,int userID,BFCPError e);

        public void gotChairAck(long conferenceID,int userID);

        public void gotChairRequest(long conferenceID,int userID,BFCPChairRequestInfo
info);
        public void managePendingChairRequest(long conferenceID,int
userID,BFCPChairRequestInfo info);
}
```

Attraverso tali metodi si riceveranno notifiche dal server riguardo:

- Ricevuto *hello*
- Ricevuta una risposta ad un `FloorRequestStatus`
- Ricevuta risposta ad una richiesta `UserInfo`
- Ricevuta risposta ad un `FloorStatus`
- Ricevuto Errore
- Ricevuta conferma ad un azione di moderatore
- Ricevuta richiesta di azione moderatore
- Gestione di richieste di moderazione pendenti

L'oggetto di tipo `BFCPChairRequestInfo` fornito come argomento di alcuni metodi è un contenitore di informazioni relative al messaggio notificato e fornisce attraverso i propri metodi il FloorRequestID e i relativi UserID e Floor associati:

```
        public int[] getFloors()

        public int getUserID()

        public int getFloorRequestId()
```

3.8 Implementazione modulo UserAgent

Il modulo UserAgent rappresenta il fulcro dell'applicativo, si collega direttamente e indirettamente a tutti i moduli preesistenti fornendo i metodi necessari all'interfaccia grafica per l'avvio della comunicazione con il server SIP e per la gestione della moderazione attraverso il server BFCP. Nell'implementazione si è deciso di dividere il processo di registrazione presso il server SIP dal processo di comunicazione con relativa gestione dei media, per questo sono state implementate due classi: `RegisterAgent` e `UserAgent`, quest'ultima classe è affiancata dalla classe: `UserAgentProfile`, una classe di supporto e configurazione che esamineremo in dettaglio successivamente.

La classe `RegisterAgent` implementa `TransactionClientListener` e `SubscriberDialogListener` ed è utilizzata per la fase di registrazione al server SIP di riferimento. Il costruttore di tale classe come argomento, oltre ad un oggetto di tipo `SipProvider` e ai dettagli di registrazione, necessita anche di un oggetto di tipo `RegisterAgentListener` che si occuperà di ricevere ed interpretare le risposte ricevute dall'istanza della classe.

L'oggetto di tipo `SipProvider` sarà utilizzato dalle istanze della classe `RegisterAgent` durante tutto il processo di comunicazione con il server SIP.

```
public RegisterAgent(SipProvider sip_provider, String target_url,
                     String contact_url, String username, String realm, String passwd,
                     RegisterAgentListener listener,UserAgentProfile user_profile)
```

I metodi di fondamentale importanza in tale classe che consentono la registrazione/deregistrazione al server SIP sono:

```
public boolean register(int expire_time) {
        [...]
        Message req = MessageFactory.createRegisterRequest(sip_provider,
                                target, target, new NameAddress(user_profile.contact_url));
        req.setExpiresHeader(new ExpiresHeader(String.valueOf(expire_time)));
        [...]
        t = new TransactionClient(sip_provider, req, this, 30000);
        t.request();
        return true;
}
public boolean unregister() { [...]
        return register(0);
}
```

Nel metodo `register(...)` viene creato un oggetto di tipo `Message` attraverso la classe *helper* `MessageFactory` e tale oggetto viene inviato attraverso un'istanza della classe `TransactionClient` con l'uso del metodo `request()`. Attraverso metodi di *callback* alla ricezione di registrazione effettuata/fallita l'oggetto di tipo `RegisterAgent` invocherà gli opportuni metodi del *listener* di tipo `RegisterAgentListener` per comunicare l'esito della registrazione.

La classe `UserAgent` estende `CallListenerAdapter` ed implementa `BFCPListener`. Il costruttore della classe `UserAgent` riceve come argomenti un oggetto di tipo `SipProvider` ed un oggetto di tipo `UserAgentProfile`.

```
public UserAgent(SipProvider sip_provider, UserAgentProfile user_profile) {
        this.sip_provider = sip_provider;
        this.user_profile = user_profile;
        [...]
}
```

L'oggetto di tipo `SipProvider` sarà utilizzato dalle istanze della classe `UserAgent` durante tutto il processo di comunicazione con il server SIP.

La classe `UserAgentProfile` è una classe di supporto e configurazione che esamineremo in dettaglio successivamente.

I metodi accessibili della classe `UserAgent` che permettono l'avvio della comunicazione multimediale attraverso il server SIP e la richiesta/moderazione di risorse attraverso il server BFCP sono: `call(...)` e `hangup(...)`.

```
public boolean call(String target_url, boolean send_anonymous) {
        [...]
        String from_url;
        if (!send_anonymous) { from_url = user_profile.from_url; } else {
                from_url = "sip:anonymous@anonymous.com";
        }
        createOffer();
        call = new ExtendedCall(sip_provider, from_url,
                                user_profile.contact_url, user_profile.username,
                                user_profile.realm, user_profile.passwd, this);
        if (target_url.indexOf("@") < 0) { target_url = target_url + "@" + realm; }
        target_url = sip_provider.completeNameAddress(target_url).toString();
        if (user_profile.no_offer) {  call.call(target_url);
       } else { call.call(target_url, local_session); }
        return true;
}
```

Il metodo `call(...)` avvia la comunicazione con relativa contrattazione dei *codec* da

utilizzare durante lo scambio di flussi multimediali con il server SIP. La creazione del messaggio SDP per iniziare il *three-way-handshaking* avviene attraverso il metodo privato `createOffer()` che aggiunge all'istanza della classe `UserAgentProfile` i *codec* disponibili dal client. Successivamente viene creata un'istanza della classe `ExtendedCall` e viene invocato il metodo `call(...)` con gli opportuni parametri. E' necessario chiarire che il processo di *three-way-handshaking* è solo iniziato in questa funzione e sarà completato attraverso i metodi di callback derivati dalla classe `CallListenerAdapter`.

```
public void hangup() {
        closeMediaApplication();
        if (call != null) {
                call.hangup();
        }
        if (Receiver.bfcpCtrl != null) {
                BFCPDispatcher.removeListener(this);
        }
        [...]
        if (Receiver.bfcpCtrl != null) {
                Receiver.bfcpCtrl.disconnect();
                Receiver.bfcpCtrl = null;
        }
}
```

Il metodo `hangup(...)` serve invece ad interrompere eventuali comunicazioni multimediali in corso attraverso l'invocazione del metodo privato `closeMediaApplication()` che interrompe l'invio di flussi multimediali verso il server SIP. Attraverso il metodo `hangup()` invocato sull'istanza della classe `ExtendedCall` comunica al server la volontà di interrompere la comunicazione. Infine il metodo `disconnect()` sull'istanza della classe `BFCPControlCenter` comunica la chiusura delle comunicazioni al server BFCP.

I metodi derivati dalla classe `CallListenerAdapter` permettono ad un oggetto di tipo `UserAgent` di gestire correttamente le notifiche ottenute, attraverso il *dispatcher*, dal server SIP. In particolare citiamo i seguenti metodi:

```
public void onCallAccepted(Call call, String sdp, Message resp)

public void onCallClosing(Call call, Message bye)

public void onCallModifying(Call call, String sdp, Message invite)
```

Il metodo `onCallAccepted(...)` viene invocato dal *dispatcher* della libreria MjSip alla risposta del server ad una richiesta di comunicazione, ciò avviene dopo che il metodo call(...) della classe `UserAgent` è stato invocato con gli opportuni parametri.

```
public void onCallAccepted(Call call, String sdp, Message resp) { [...]
        SessionDescriptor remote_sdp = new SessionDescriptor(sdp);
        if (user_profile.no_offer) {
                createAnswer(remote_sdp);
                call.ackWithAnswer(local_session);
        } else {
                sessionProduct(remote_sdp);
        }
        launchMediaApplication();
}
```

Innanzitutto viene creato un nuovo oggetto di tipo `SessionDescriptor` con il messaggio SDP ricevuto, successivamente se nell'istanza della classe `UserAgentProfile` non c'era già un'offerta la si crea a partire da quella remota con il metodo `createAnswer(...)` e la si invia, in caso contrario, se c'era già un'offerta, allora si procede all'aggiornamento di quella esistente a partire da quella remota con il metodo privato `sessionProduct(...)`. Infine si avvia la trasmissione/ricezione dei flussi multimediali attraverso il metodo protetto `launchMediaApplication()`.

```
protected void launchMediaApplication() {
        [...]
        Codecs.Map c;
        c = Codecs.getCodec(local_sdp);
        [...]
        audio_app = new JaudioLauncher(local_audio_port, remote_media_address,
                remote_audio_port, dir, audio_in, audio_out, c.codec.samp_rate(),
                user_profile.audio_sample_size, c.codec.frame_size(), log, c, dtmf_pt);
        [...]
        audio_app.startMedia();
        [...]
}
```

Analizzando in dettaglio il metodo protetto `launchMediaApplication()` si può vedere come istanzia un'oggetto di tipo `JaudioLauncher` descritto precedentemente e avvia la trasmissione dei flussi multimediali.

Il metodo `onCallClosing(...)` viene invocato dal *dispatcher* della libreria MjSip alla risposta del server ad una richiesta di chiusura di comunicazione attraverso il metodo `hangup()` della classe.

```
public void onCallClosing(Call call, Message bye) {
        [...]
        closeMediaApplication();
[...] }
```

Tale *callback* si preoccupa di interrompere la trasmissione dei flussi multimediali attraverso il metodo protetto closeMediaApplication().

```
protected void closeMediaApplication() {
        if (audio_app != null) {
                audio_app.stopMedia();
                audio_app = null;
        }
}
```

Il metodo protetto `closeMediaApplication()` provvede a interrompere la trasmissione/ricezione di flussi multimediali tramite il server SIP invocando il metodo `stopMedia()` dell'istanza della classe `JaudioLauncher`.

Il metodo `onCallModifying` viene invocato alla ricezione di un *re-invite* da parte del server al fine di modificare i dettagli della comunicazione corrente. Tale metodo viene chiamato dal *dispatcher* del server SIP che fornisce come argomento l'istanza di riferimento della classe `Call` attraverso la quale è possibile reperire il messaggio SDP remoto da cui estrapolare i dettagli per la comunicazione con il server BFCP.

```
public void onCallModifying(Call call, String sdp, Message invite) {
        [...]
        super.onCallModifying(call, sdp, invite);
        SdpParser sdpP = new SdpParser(call.getRemoteSessionDescriptor());
        sdpP.parseReinviteAttributes();

        String _userid = sdpP.getBindings().get("userid");
        String _confid = sdpP.getBindings().get("confid");
        String _port = sdpP.getBindings().get("port");

        if (_userid != null && _confid != null && _port != null) {
                Receiver.bfcpVector = new HashMap<String, String>();

                Receiver.bfcpVector.put("port", _port);
                Receiver.bfcpVector.put("userid", _userid);
                Receiver.bfcpVector.put("confid", _confid);
                Receiver.bfcpVector.put("audioLabel",sdpP.getBindings().get("audioLabel"));
                Receiver.bfcpVector.put("videoLabel",sdpP.getBindings().get("videoLabel"));
                Receiver.bfcpVector.put("audioFloorid", sdpP.getBindings().get(
                                        "audioFloorid"));
                Receiver.bfcpVector.put("videoFloorid", sdpP.getBindings().get(
                                        "videoFloorid"));
                Receiver.bfcpCtrl = new BFCPControlCenter(
                                        Integer.parseInt(_userid), Long.parseLong(_confid));

                BFCPDispatcher.addListener(this);

                try {
                        Receiver.bfcpCtrl.connect(Receiver.serverIpAddr, Integer
                                                .parseInt(_port));
                        Receiver.bfcpCtrl.helloServer();
                } catch (KeyManagementException e) {
                        e.printStackTrace();
                } catch (UnknownHostException e) {
                        e.printStackTrace();
                } catch (IOException e) {
                        e.printStackTrace(); } } }
```

Viene istanziato un oggetto di tipo `SdpParser` attraverso il quale viene invocato il metodo `parseReinviteAttributes(...)` con cui si effettua l'eventuale *parsing* dei dettagli per la comunicazione con il server BFCP. Se il *parsing* fornisce esito positivo viene creato un vettore con i dettagli appena acquisiti e successivamente viene invocato il metodo `addListener(...)` della classe statica `BFCPDispatcher` che aggiunge l'istanza della classe `UserAgent` alla lista dei *listener* da avvisare in caso di notifiche da parte del server BFCP. Successivamente viene avviata la connessione e viene iniziata la

comunicazione con il metodo `helloServer()`.

I metodi implementati dell'interfaccia pubblica `BFCPListener` permettono ad un oggetto di tipo `UserAgent` di gestire correttamente le notifiche ottenute, attraverso il *dispatcher*, dal server BFCP. Analizziamo quindi i seguenti metodi:

```
public void gotHello(long conferenceID, int userID)
public void gotFloorRequestStatus(long conferenceID, int userID, BFCPFloorRequestInfo
details)
public void gotChairRequest(long conferenceID, int userID, BFCPChairRequestInfo info)
public void managePendingChairRequest(long conferenceID,int userID,BFCPChairRequestInfo
info)
public void gotFloorStatus(long conferenceID, int userID, int floor)
public void gotUserStatus(long conferenceID, int userID, String user)
```

Il metodo gotHello(...) viene invocato dal *dispatcher* alla ricezione di risposta ad un messaggio di "*hello*" da parte del server BFCP. Tale messaggio indica all'istanza della classe UserAgent che la comunicazione è iniziata correttamente e procede alla verifica delle risorse moderate interrompendo le rispettive trasmissioni multimediali.

```
public void gotHello(long conferenceID, int userID) {
        if (!gotHello) {
                gotHello = true;
                if(Receiver.bfcpVector.get("audioFloorid")!=null){
                        muteMediaApplication();
                        ((ManagerUI) Receiver.mContext).displayMessage("Conferenza Moderata
                                con BFCP. (muto)", false);
                }
        }
}
```

Nello specifico il metodo dopo aver verificato la presenza delle risorse moderate, prima di notificare l'evento all'utente attraverso il gestore dell'interfaccia grafica, chiama il metodo pubblico `muteMediaApplication()` che si preoccupa di comunicare all'istanza della classe `JaudioLauncher` di interrompere la trasmissione in uscita dei flussi multimediali.

```
public boolean muteMediaApplication() {
        if (audio_app != null)
                return audio_app.muteMedia();
        return false;
}
```

Il metodo `gotFloorRequestStatus()` viene utilizzato per gestire il cambio di status di una risorsa richiesta dall'utente. In base a tale cambiamento di status questo metodo regola l'interruzione e/o l'avvio della trasmissione dei flussi multimediali.

```
public void gotFloorRequestStatus(long conferenceID, int userID,BFCPFloorRequestInfo
details) {
	[...]
	int reqStatus = details.getStatus();
	boolean statusChanged = false;
	for (int i = 0; i < floors.length; i++) {
		if (floors[i] == audioFloorid) {
			if (bfcpAudioStatus != (short) reqStatus) {
				[...]
				bfcpAudioStatus = (short) reqStatus;
				statusChanged = true;
				switch (reqStatus) {
					case FloorRequest.CANCELLED:
						muteMediaApplication();
						break;
					case FloorRequest.GRANTED:
						muteMediaApplication();
						break;
					case FloorRequest.RELEASED:
						muteMediaApplication();
						break;
					case FloorRequest.REVOKED:
						muteMediaApplication();
						break;
				}
			}
		}
	[...]
	if (statusChanged) {
		sendMsg(reqStatus, msg);
	}
}
```

Analizzando in dettaglio tale funzione si può verificare che viene prelevato lo status ricevuto della risorsa attraverso l'oggetto di tipo `BFCPFloorRequestInfo`, successivamente se lo status è cambiato rispetto a quello precedente allora si procede all'abilitazione/disabilitazione della trasmissione dei flussi multimediali attraverso il metodo `muteMediaApplication()` già visto in dettaglio precedentemente, successivamente si invoca il metodo privato `sendMsg()` con lo status della risorsa e il messaggio da inviare all'*Activity* che sta gestendo l'interfaccia grafica, in modo da notificare all'utente il cambiamento.

Il metodo `gotChairRequest(...)` viene utilizzato per gestire eventuali richieste di risorsa nella modalità moderatore. Tali richieste saranno inviate dal server BFCP solo dopo che il client si sarà autenticato correttamente attraverso il protocollo CCMP e avrà inviato

la notifica al server BFCP attraverso il metodo `beTheChair(...)` dell'istanza della classe `BFCPControlCenter`.

```
public void gotChairRequest(long conferenceID, int userID,
                        BFCPChairRequestInfo info) {
        [...]
        ArrayList<BFCPChairRequestInfo> requests = new ArrayList<BFCPChairRequestInfo>();
        requests.add(info);
        Receiver.bfcpCtrl.getPendingChairRequests().put(info.getUserID(), requests);
        [...]
}
```

Analizzando in dettaglio tale metodo si può notare che al momento della ricezione di una *ChairRequest* tale richiesta viene aggiunta ad una struttura dati dell'istanza della classe `BFCPControlCenter` per permettere alla libreria di reperire le informazioni sull'utente che ha effettuato tale richiesta. La richiesta sarà poi gestita dall'istanza della classe UserAgent con l'invocazione del metodo `managePendingChairRequest(...)`.

Il metodo `managePendingChairRequest(...)` viene utilizzato per gestire *ChairRequest* pendenti aggiunte alla struttura dati interna all'istanza del BFCPControlCenter.

```
public void managePendingChairRequest(long conferenceID, int userID,
                        BFCPChairRequestInfo info) {
        int[] _chairReqDetails = new int[] { 0, 0, 0, 0, 0 };
        _chairReqDetails[0] = (int) conferenceID;
        _chairReqDetails[1] = info.getUserID();
        int[] _floors = info.getFloors();
        [...]
        _chairReqDetails[4] = info.getFloorRequestId();
        ((mainSipCon) Receiver.mContext).displayChairRequest(_chairReqDetails);
}
```

Analizzando in dettaglio si nota che viene costruito un *array* di interi con le informazioni relative alla richiesta e tale *array* viene passato alla Activity che gestisce l'interfaccia grafica per notificare all'utente tale richiesta.

Il metodo `gotFloorStatus(...)` viene invocato dal *dispatcher* della libreria BFCP alla ricezione di un *FloorStatus*, cioè un aggiornamento sullo status di un floor, tale metodo viene quindi utilizzato dall'istanza della classe `UserAgent` per notificare eventuali cambiamenti di stato delle risorse richieste all'interfaccia grafica nel caso in cui si sia avviata la modalità moderatore.

```
public void gotFloorStatus(long conferenceID, int userID, int floor) {
        HashMap<Integer, String[]> _hm = new HashMap<Integer, String[]>();
        for (FloorInfo _floorInfo : Receiver.bfcpCtrl.getObservedFloorList()) {
                for (FloorRequest _floorRequest : _floorInfo.getRequests()) {
                        if (_floorRequest.getBeneficiaryID() != userID) {
                        String _username = getNameById(_floorRequest.getBeneficiaryID());
                        if (_username == null) {
                        Receiver.bfcpCtrl.userInfo(_floorRequest.getBeneficiaryID());
                        _username = "Sconosciuto";
                        }
                        }
                        [...]
                }
        }
        ((mainSipCon) Receiver.mContext).updateUsersList(_hm);
}
```

Per ogni istanza della classe `FloorInfo` fornita dal metodo `getObservedFloorList()` della classe `BFCPControlCenter` si vanno ad analizzare tutte le istanze della classe `FloorRequest` creando la struttura dati da inviare all'Activity che gestisce l'interfaccia grafica permettendo l'aggiornamento della lista degli utenti che hanno fatto richiesta di una risorsa. Tale struttura dati viene creata con le sole richieste il cui `userID` non corrisponda a quello proprio, ovvero del moderatore. Nel caso in cui il nome utente corrispondente all'id utente analizzato non sia presente nella struttura dati interna all'istanza della classe `BFCPControlCenter` allora si invoca il metodo `userInfo(...)` che richiede asincronamente informazioni sull'utente con `userID` passato come argomento. La risposta al metodo asincrono `userInfo(...)` verrà gestita dal metodo `gotUserStatus(...)`.

```
public void gotUserStatus(long conferenceID, int userID, String benID) {
        if (Receiver.bfcpCtrl.getParticipantsMap() != null
                && Receiver.bfcpCtrl.getParticipantsMap().get(Integer.parseInt(benID)) !=
        null && Receiver.bfcpCtrl.getParticipantsMap().get(Integer.parseInt(benID))
                .getUserDisplayName() != null) {

                ((mainSipCon) Receiver.mContext).updateUsername(Integer
                        .parseInt(benID),Receiver.bfcpCtrl.getParticipantsMap()
                        .get(Integer.parseInt(benID)).getUserDisplayName());

                }
        [...]
}
```

Il metodo `gotUserStatus(...)` dopo aver verificato che le informazioni dell'id utente passato come argomento `benID` sono state recuperate si occupa di notificare all'Activity responsabile dell'interfaccia grafica che è necessario aggiornare l'*username* con il dato *userid* nella lista degli utenti richiedenti risorse.

L'activity responsabile dell'interfaccia grafica comunicherà con l'istanza della classe `UserAgent` attraverso i seguenti metodi:

```
public void setChair(String passwd, boolean audioM, boolean videoM)
public void acceptFloorRequestId(int _floorRequestId)
public void denyFloorRequestId(int _floorRequestId)
public void revokeFloorRequestId(int _floorRequestId, int[] floors)
public void removeChair()
```

Il metodo `setChair(...)` viene invocato dall'Activity responsabile della gestione dell'interfaccia grafica per avviare attraverso l'istanza della classe `UserAgent` il processo di autenticazione al fine di abilitare l'utente alla moderazione della conferenza.

```
public void setChair(String passwd, boolean audioM, boolean videoM) {
if (getBFCPstatus()) {
        [...]
        bfcpPasswd = passwd;
        BFCPmod bfcpmod = new BFCPmod(Receiver.serverIpAddr,
                Receiver.portBFCPScheduler, (String) Receiver.bfcpVector
                        .get("confid"), (String) Receiver.bfcpVector
                                .get("userid"), floorsArray, passwd, this);

        bfcpmod.start();
        }
[...]
}
```

La password viene salvata per la successiva rimozione della moderazione e viene avviato il *thread* del tipo `BFCPmod` che avvia il processo di autenticazione presso il server BFCP attraverso il protocollo CCMP. L'istanza della classe UserAgent sarà avvisata dal *thread* di tipo `BFCPmod` attraverso il metodo `gotChairReply(...)` che fornisce come argomento i *floor* per cui si è diventato moderatore.

```
public void gotChairReply(int[] floors) {
        Receiver.bfcpCtrl.beTheChair(floors);
}
```

Tale funzione si occuperà di comunicare al server BFCP attraverso il metodo `beTheChair(...)` della classe `BFCPControlCenter` di abilitare l'invio delle notifiche

di moderazione.

Il metodo `acceptFloorRequestId()` viene invocato per accettare la richiesta di risorsa con *FloorRequestID* specificato come argomento. Tale metodo invoca `acceptFloorRequest(...)` sull'istanza della classe `Chair` dell'oggetto `bfcpCtrl`.

```
public void acceptFloorRequestId(int _floorRequestId) {
	Receiver.bfcpCtrl.getChair().acceptFloorRequest(_floorRequestId);
}
```

Il metodo `denyFloorRequestId()` viene invocato per rifiutare la richiesta di risorsa con *FloorRequestID* specificato come argomento. Tale metodo invoca `denyFloorRequest(...)` sull'istanza della classe `Chair` dell'oggetto `bfcpCtrl`.

```
public void denyFloorRequestId(int _floorRequestId) {
	Receiver.bfcpCtrl.getChair().denyFloorRequest(_floorRequestId);
}
```

Il metodo `revokeFloorRequestId()` viene invocato per revocare la richiesta di risorsa precedentemente accettata con *FloorRequestID* specificato come argomento. Tale metodo invoca `revokeFloorRequest(...)` sull'istanza della classe `Chair` dell'oggetto `bfcpCtrl`. Nel caso in cui il metodo ritorni un intero pari al valore "`-1`" allora tale metodo invoca `revokeFloorRequest(...)` passandogli come argomento oltre al *floorRequestID* anche un'array di *floor*. Questa procedura è necessaria in quanto il primo metodo ritorna valore "`-1`" nel caso in cui il *floorRequestID* non è presente nella lista della richieste ricevute, tuttavia un utente potrebbe aver ottenuto una risorsa prima che il moderatore abbia effettuato l'accesso, pertanto è necessario che l'istanza della classe `UserAgent` invochi il metodo alternativo fornendo come argomento aggiuntivo la lista dei *floor* da revocare.

```
public void revokeFloorRequestId(int _floorRequestId, int[] floors) {
	if (Receiver.bfcpCtrl.getChair().revokeFloorRequest(_floorRequestId) == -1) {
		Receiver.bfcpCtrl.getChair().revokeFloorRequest(_floorRequestId, floors);
	}
}
```

Il metodo `removeChair()` viene invocato per disabilitare la moderazione precedentemente acquisita. La disabilitazione avviene fornendo al costruttore della classe BFCPmod come argomento `userid` pari a "`0`".

```
public void removeChair() {
        [...]
        BFCPmod bfcpmod = new BFCPmod(Receiver.serverIpAddr,
                Receiver.portBFCPScheduler, (String) Receiver.bfcpVector
                        .get("confid"), "0", floorsArray, bfcpPasswd, this);

        bfcpmod.start();
        Receiver.bfcpCtrl.beTheChair(false);
        bfcpAudioMod=false;
        bfcpAudioMod=false;
}
```

3.9 Implementazione dell'interfaccia grafica

L'implementazione dell'interfaccia grafica è stata effettuata in parte per via visuale grazie all'uso degli strumenti forniti dal *plugin* per Eclipse. Tali strumenti permettevano la creazione di interfacce grafiche per ambiente Android che venivano poi convertite in appositi file XML (*eXtensible Markup Language*) gestiti dall'applicativo. Una parte del file XML utilizzato per l'interfaccia grafica è riportato di seguito:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="fill_parent" android:layout_height="fill_parent"
android:padding="10px" android:orientation="vertical">
<RelativeLayout android:id="@+id/RelativeLayout01"
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_height="wrap_content" android:layout_width="wrap_content"
android:gravity="center_horizontal" android:layout_gravity="center_horizontal"
android:layout_marginBottom="15px">

<TextView android:layout_width="wrap_content" android:id="@+id/Titolo"
android:layout_height="wrap_content" android:text="@string/hello"
android:layout_centerHorizontal="true" android:textSize="18px"></TextView>
<EditText android:layout_centerHorizontal="true" android:id="@+id/EditNumber"
android:layout_below="@+id/Titolo" android:layout_height="wrap_content"
android:layout_width="wrap_content" android:width="180px" android:layout_marginTop="15px"
android:text="8971751" android:inputType="numberDecimal" android:lines="1"></EditText>

<Button android:id="@+id/ButtonChiama" android:layout_below="@+id/EditNumber"
android:layout_height="wrap_content" android:layout_width="wrap_content"
android:layout_centerHorizontal="true" android:background="@drawable/dial_answer"
android:layout_marginTop="10px"></Button>
[...]
</RelativeLayout>
<ListView android:layout_width="wrap_content" android:layout_height="wrap_content"
android:id="@+id/ListaUtenti"></ListView>
</LinearLayout>
```

La programmazione testuale si è invece occupata dell'interazione dell'utente con tale interfaccia grafica definita in file XML. La classe che si occupa della gestione dell'interfaccia grafica occupandosi dei pulsanti e del salvataggio degli stati è `mainSipCon`. Tale classe estende la classe `Activity` e implementa le interfacce pubbliche `RegisterAgentListener` e `ManagerUI`.

L'estensione della classe Activity permette l'*override* dei metodi utilizzati per la creazione dell'interfaccia grafica inizializzando le variabili globali e creando i *listener* necessari alla gestione dei pulsanti, al fine di garantire una corretta interattività con l'utente.

```
@Override
public void onCreate(Bundle savedInstanceState) {
                super.onCreate(savedInstanceState);
                setContentView(R.layout.main);

                listViewUtenti = (ListView) findViewById(R.id.ListaUtenti);
                buttonChiama = (Button) findViewById(R.id.ButtonChiama);
                buttonPausa = (Button) findViewById(R.id.ButtonMuto);
                buttonVivavoce = (Button) findViewById(R.id.ButtonVivavoce);
                buttonBFCPmod = (Button) findViewById(R.id.ButtonBFCPMod);
                buttonStartAudio = (Button) findViewById(R.id.ButtonBFCPAudio);
                et = (EditText) findViewById(R.id.EditNumber);
                customAdapterUtenti=null;
                listaUtenti = new ArrayList<String[]>();
                [...]
                initSip();
                [...]
}
```

All'interno del metodo `onCreate(...)` si ottengono i riferimenti a tutti gli elementi dell'interfaccia grafica e si inizializzano le variabili globali, si invoca inoltre il metodo privato `initSip()` che si occupa di inizializzare il modulo SIP per la corretta registrazione e connessione al server SIP di riferimento.

```
private void initSip(){
        Receiver.mContext = this;
        String username = Receiver.username;
        String passwd = Receiver.passwd;
        String version = Receiver.version;
        SipStack.init(null);
        SipStack.debug_level = 0;
        SipStack.max_retransmission_timeout = 4000;
        SipStack.transaction_timeout = 30000;
        SipStack.default_transport_protocols = new String[1];
        SipStack.default_transport_protocols[0] = "udp";
        SipStack.default_port = 5060;
        SipStack.ua_info = version;
        SipStack.server_info = version;
        IpAddress.setLocalIpAddress();
        String ipaddr = IpAddress.localIpAddress;

        SipProvider sip_provider = new SipProvider(ipaddr, 0);
```

```
        String contact_url = username + "@" + ipaddr + ":"
                                + sip_provider.getPort() + ";transport=udp";

        UserAgentProfile user_profile = new UserAgentProfile(null);
        String target_url = Receiver.serverIpAddr;
        user_profile.username = username;
        user_profile.passwd = passwd;
        user_profile.realm = target_url;
        user_profile.contact_url = contact_url;
        user_profile.from_url = username + "@" + target_url;
        ua = new UserAgent(sip_provider, user_profile);
        ra = new RegisterAgent(sip_provider, user_profile.from_url,
                        user_profile.contact_url, user_profile.username,
                        user_profile.realm, user_profile.passwd, mainSipCon.this,
                        user_profile);
        }
```

Nel metodo `initSip()` innanzitutto vengono recuperati i dettagli dalla classe `Receiver`, successivamente viene inizializzato lo *stack* SIP attraverso la classe statica `SipStack` quindi vengono istanziati gli oggetti di tipo `UserAgentProfile`, `UserAgent` e `RegisterAgent`.

All'interno del metodo `onCreate(...)` vengono inoltre creati i *listener* che si occuperanno di intercettare gli eventi generati dalla pressione di un dato pulsante da parte dell'utente.

Ecco di seguito la definizione e l'aggiunta del *listener* al pulsante dedicato all'avvio/interruzione della comunicazione:

```
buttonChiama.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
                synchronized (mutex) {
                if (ra == null || !ra.isRegistered()) {
                        Receiver.confid = et.getText().toString();
                        ra.register();
                        ua.listen();
                        ua.call(Receiver.confid + "@" + Receiver.serverIpAddr, false);
                        buttonChiama.setBackgroundResource(R.drawable.dial_decline);
                        displayMessage("Chiamata in corso..");
                } else {
                        buttonChiama.setBackgroundResource(R.drawable.dial_answer);
                        halt();
                        displayMessage("Chiamata terminata!");
                }
                }
        }
});
```

Se l'istanza della classe `RegisterAgent` non ha ancora effettuato la registrazione presso il server SIP, il metodo onClick(...) del *listener* provvede a registrarsi e ad avviare la comunicazione invocando il metodo `call(...)` sull'istanza della classe `UserAgent`. In caso contrario interrompe la comunicazione invocando il metodo privato `halt()`. Il metodo `halt()` interrompe la comunicazione invocando gli appositi metodi sulle istanze della classi responsabili della comunicazione. Di seguito è mostrata una parte del codice implementato:

```
public void halt() {
        if (ua != null)
                ua.hangup();
        if (ra != null) {
                ra.halt();
                ra.unregister();
        }
        if(customAdapterUtenti!=null){
                listaUtenti.clear();
                customAdapterUtenti.changeDataSet(listaUtenti);
                customAdapterUtenti.notifyDataSetChanged();
        }
        buttonPausa.setBackgroundResource(R.drawable.dial_answer_and_hold);
        buttonVivavoce.setBackgroundResource(R.drawable.sound_on);
        buttonStartAudio.setBackgroundResource(R.drawable.microphone);
}
```

L'interfaccia pubblica `ManagerUI` è stata creata per generalizzare i metodi usati per la gestione delle interazioni con l'interfaccia grafica di riferimento.

```
public interface ManagerUI {
        public void displayMessage(String str);

        public void displayMessage(String str, boolean um);

        public void updateUsersList(HashMap<Integer, String[]> _hm);

        public void updateUsername(int _id,String _username);

        public void displayChairRequest(int[] _chairRequestDetails);
}
```

Iniziando ad analizzare il primo metodo: `displayMessage(str)`, l'implementazione è stata incentrata in modo che ogni interazione con l'interfaccia grafica, a partire da *thread* diversi da quello che si sta occupando della gestione, deve necessariamente passare per un *handler* istanziato dal gestore stesso dell'interfaccia grafica. Questa che potrebbe sembrare una limitazione dell'ambiente Android in realtà garantisce un codice più chiaro e pulito evitando ambiguità.

```
public void displayMessage(String str) {
        android.os.Message _mess = new android.os.Message();
        _mess.what = DISPLAY;
        Bundle _b = new Bundle();
        _b.putString(String.valueOf(_mess.what), str);
        _mess.setData(_b);
        uiUpdateHandler.sendMessage(_mess);
}
```

La comunicazione con l'*handler* avviene mediante lo scambio di messaggi, l'*handler* in questione gestisce più modalità operazionali, per tale motivo si segnala il tipo di operazione come intero nel campo `what` dell'istanza della classe `android.os.Message`.

Prima di analizzare gli altri metodi dell'interfaccia pubblica `ManagerUI`, diamo uno sguardo alla struttura dell'*handler*:

```
Handler uiUpdateHandler = new Handler() {
        public void handleMessage(android.os.Message m) {
                int _wht = m.what;
                switch (_wht) {
                case DISPLAY:
                        String _msg = m.getData().getString(String.valueOf(_wht));
                        Toast.makeText(getApplicationContext(), _msg,
                                Toast.LENGTH_SHORT).show();
                        break;
                case UNMUTE:
                        boolean _unmute = m.getData().getBoolean(String.valueOf(_wht));
                        if (_unmute) {
                        buttonStartAudio.setBackgroundResource(R.drawable.microphone);
                        } else {
                        buttonStartAudio.setBackgroundResource(R.drawable.microphoneoff);
                        }
                        break;
                case CHAIR_REQ:
                        int[] _chairRequestDetails = m.getData().getIntArray(
                                String.valueOf(_wht));
                        System.out.println("Visualizzo l'alertDialog!");
                        alertDialogChairRequest(_chairRequestDetails);
                        break;
                case REFRESH_LIST:
                        if (customAdapterUtenti==null){
                        customAdapterUtenti = new
                                CustomAdapter(Receiver.mContext,listaUtenti,ua);
                        listViewUtenti.setAdapter(customAdapterUtenti);
                        }else{
                        ((CustomAdapter)listViewUtenti.getAdapter())
                                .changeDataSet(listaUtenti);
                        ((CustomAdapter)listViewUtenti.getAdapter())
                                .notifyDataSetChanged();
                        }
                        break;
                case EDIT_USERNAME:
                        String[] _userDetails = m.getData().getStringArray(
                                String.valueOf(_wht));
                        CustomAdapter adapter =(CustomAdapter) listViewUtenti.getAdapter();
                        for (int i=0; i< adapter.getCount();i++){
                                String[] value= (String[])adapter.getItem(i);
                                if (value[0].equalsIgnoreCase(_userDetails[0])){
                                        value[1]=_userDetails[1];
                                        adapter.changeValue(i, value);
                                        adapter.notifyDataSetChanged();
                                }
                        }
                        break;
                }
        super.handleMessage(m);
        }
};
```

Andando ad analizzare il costrutto `switch-case` nell'implementazione dell'*handler* il messaggio ricevuto dall'invocazione del metodo `displayMessage(str)` attraverso l'uso della classe Toast, gestente notifiche a comparsa tipiche dell'ambiente Android.

Il metodo `displayMessage(str,um)` oltre a richiamare il metodo precedente con il messaggio da visualizzare, invia all'*handler* anche un valore booleano corrispondente al valore corrente della funzionalità `UNMUTE` del modulo *GestioneMedia*. L'istanza della classe `Handler` alla ricezione di tale messaggio, provvederà quindi a cambiare stato al pulsante corrispondente.

Analizziamo adesso il metodo `updateUsersList(...)`, tale metodo viene invocato dall'istanza della classe `UserAgent` alla ricezione di un `FloorStatus`, cioè un aggiornamento sullo stato delle risorse assegnate.

```
public void updateUsersList(HashMap<Integer, String[]> _hm){
        android.os.Message _mess = new android.os.Message();
        _mess.what = REFRESH_LIST;
        listaUtenti.clear();
        for (Integer _key : _hm.keySet()){
                listaUtenti.add(new String[]{String.valueOf(_key),_hm.get(_key[0],
                  _hm.get(_key)[1],_hm.get(_key)[2],_hm.get(_key)[3],_hm.get(_key)[4]});
        }
uiUpdateHandler.sendMessage(_mess);
}
```

In tale metodo l'istanza della classe `android.os.Message` viene utilizzata solo per la specifica del campo `what`, la struttura dati `List<String[]>` viene invece passata all'istanza della classe `Handler` come variabile globale. Tale struttura dati viene creata a partite dall'istanza della classe `HashMap` ricevuta, alla fine di tale processo il messaggio viene inviato invocando il metodo `sendMessage(_mess)` sull'oggetto di tipo `Handler`.

Il metodo `updateUsername(...)` viene invocato dall'istanza della classe `UserAgent` alla ricezione di un `UserStatus`, cioè un aggiornamento sull'utente con determinato `userid`, in genere tale aggiornamento viene notificato dal server BFCP alla richiesta esplicita di dettagli aggiuntivi su un determinato `userid`, quali ad esempio: *floor* assegnati, *username* e così via.

```
public void updateUsername(int _id,String _username){
        android.os.Message _mess = new android.os.Message();
        _mess.what = EDIT_USERNAME;
        Bundle _b = new Bundle();
        _b.putStringArray(String.valueOf(_mess.what), new String[]
                {String.valueOf(_id),_username});
        _mess.setData(_b);
        uiUpdateHandler.sendMessage(_mess);
}
```

In questo caso i valori ricevuti vengono passati all'istanza della classe `Handler` direttamente nell'oggetto di tipo `android.os.Message`, tali dettagli verranno prelevati e saranno utilizzati per l'aggiornamento della lista utenti moderati.

Analizziamo infine il metodo `displayChaiRequest(...)`, tale metodo viene invocato dall'istanza della classe `UserAgent` alla ricezione di una *chairRequest*, contenente i dettagli dell'utente che ha richiesto al server BFCP una risorsa.

```
public void displayChairRequest(int[] _chairRequestDetails) {
        android.os.Message _mess = new android.os.Message();
        Bundle _b = new Bundle();
        _mess.what = CHAIR_REQ;
        _b.putIntArray(String.valueOf(_mess.what), _chairRequestDetails);
        _mess.setData(_b);
        uiUpdateHandler.sendMessage(_mess);
}
```

Tale metodo invia i dettagli della richiesta all'istanza della classe handler attraverso l'oggetto di tipo mess. L'istanza della classe handler si occuperà quindi di invocare con gli opportuni argomenti il metodo alertDialogChairRequest.

Il metodo `alertDialogChairRequest(...)` costruisce un'istanza della classe `AlertDialog` con i dettagli della richiesta `ChairRequest`, tale istanza della classe `AlertDialog` viene quindi mostrata all'utente che può decidere attraverso tre appositi pulsanti se: "Accettare", "Rifiutare" o "Ignorare" tale richiesta da parte dell'utente mittente.

```
public void alertDialogChairRequest(int[] _chairRequestDetails) {
        Context c = Receiver.mContext;
        AlertDialog.Builder alert = new AlertDialog.Builder(c);
        final int _floorRequestId=_chairRequestDetails[4];
        final int _id = _chairRequestDetails[1];
        alert.setTitle("Chair Request");
        alert.setMessage("L'utente "+ ua.getNameById(_chairRequestDetails[1])
                + " con id: "+ _chairRequestDetails[1]+ " richiede le seguenti risorse: '"
                + (_chairRequestDetails[2] != 0 ? "Audio": "")
                +((_chairRequestDetails[2] != 0 && _chairRequestDetails[3] != 0) ? "/": "")
                + (_chairRequestDetails[3] != 0 ? "Video": "")
                + "'\nper la conferenza numero: "+ _chairRequestDetails[0]);
                alert.setPositiveButton("Accetta",new DialogInterface.OnClickListener() {
                        public void onClick(DialogInterface dialog, int whichButton)
                        {
                                ua.acceptFloorRequestId(_floorRequestId);
                        }
                });
                alert.setNegativeButton("Nega", new DialogInterface.OnClickListener() {
                        public void onClick(DialogInterface dialog, int whichButton) {
                                ua.denyFloorRequestId(_floorRequestId);
                        }
                });

                alert.setNeutralButton("Ignora", new DialogInterface.OnClickListener() {
                        public void onClick(DialogInterface dialog, int whichButton) {
                }
        });
        alert.show();
}
```

Come si può notare alla pressione di un determinato pulsante, l'istanza della classe `AlertDialog` provvederà ad invocare l'apposito metodo sull'istanza della classe `UserAgent`.

Capitolo 4

Sperimentazione

In questo capitolo saranno riportati i risultati della sperimentazione dell'applicativo in ambiente Android sia su dispositivo reale, sia sull'emulatore. Inoltre saranno presentate le principali differenze tra l'emulatore e il dispositivo reale oltre ad una breve panoramica sugli strumenti di debugging messi a disposizione dall'ambiente Android, sia sul dispositivo che sull'emulatore.

4.1 Emulatore di ambiente Android

Android SDK include un emulatore di un dispositivo mobile — un dispositivo virtuale utilizzabile su differenti sistemi operativi quali: Microsoft Windows, Gnu/linux, Mac OSX. Permette di prototipare, sviluppare, e testare applicazioni Android senza usare un vero dispositivo.

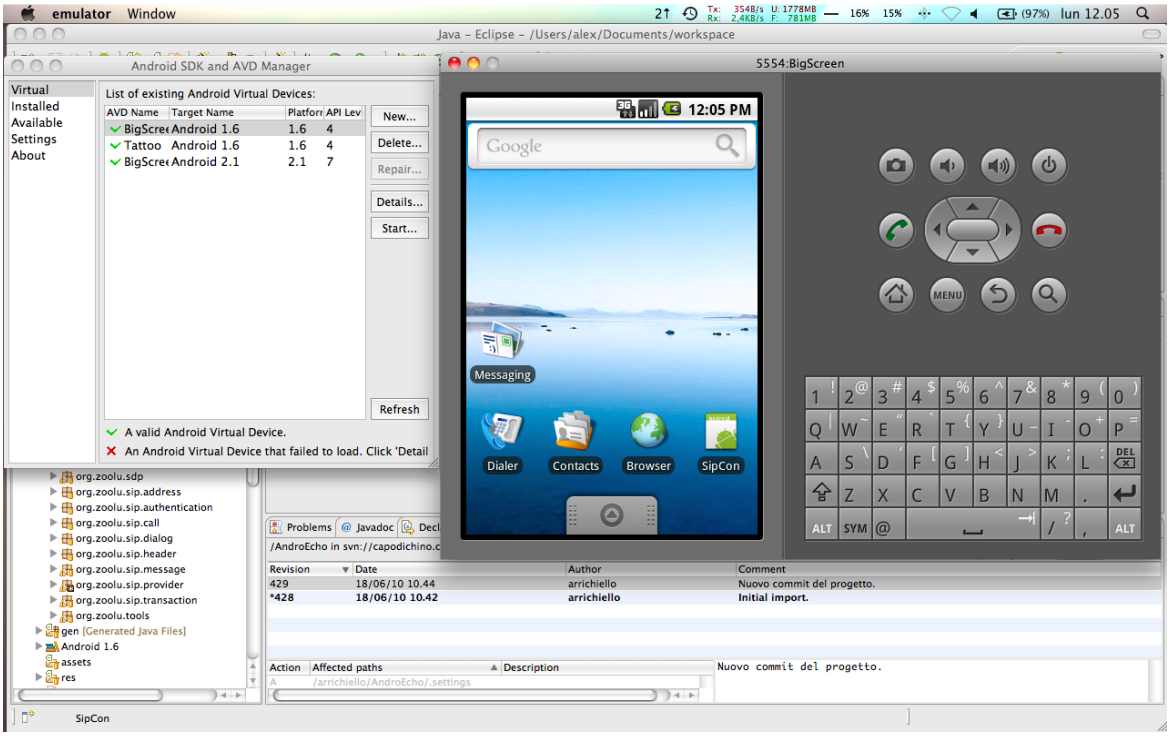


Figura 4.1: ADV Manager - Emulatore di ambiente Android

L'emulatore Android simula tutte le tipiche funzioni e i comportamenti di un dispositivo mobile, tranne il fatto che non può effettuare o ricevere normali chiamate. L'emulatore è

fornito di vari pulsanti e sistemi di interazione, che possono essere premuti utilizzando il mouse o la tastiera per generare eventi all'interno delle applicazioni testate.

Per aiutare il Design e il Testing dell'applicazione, l'emulatore permette alla stessa di usare i servizi della piattaforma Android per invocare altre applicazioni, accedere alla rete, eseguire audio e video, immagazzinare e recuperare dati, avvisare l'utente e interpretare transizioni grafiche e temi.

L'emulatore include anche una vasta gamma di funzioni per il debugging, come una console da cui visualizzare output del *kernel*, simulare interruzioni dell'applicazione (come l'arrivo di un SMS o di una chiamata), e simulare effetti di latenza e *dropouts* sul canale dei dati.

Oltre all'emulatore per studiare il comportamento di un'applicazione sviluppata si può utilizzare anche il tool: DDMS (*Dalvik Debug Monitor Service*) che è in grado di mostrare *stack* e *heap*, mettere in pausa i *thread* attivi e di esplorare il *filesystem* di ogni *device* Android connesso.

La prospettiva in Eclipse "DDMS" permette di accedere a registrazioni dello schermo dell'emulatore e la consultazione dei *log* generati dal tool "*logcat*" presente in ambiente Android.

4.2 Android Debug Bridge (ADB)

ADB: Android Debug Bridge è un'applicazione in grado di connettersi ad un *device* Android (virtuale o no) permettendo: installazione applicazioni, inviare-ricevere file, eseguire comandi di *shell scripting* sul *device*. Usando poi la *shell* del *device* è possibile cambiare impostazioni di *logging*, *query* al database SQLite, etc.

ADB è composto da tre componenti:

- Un demone avviato sull'emulatore (o *device*).
- Un servizio avviato sulla macchina di sviluppo (sul computer).
- Un *client* che permette la comunicazione con il demone attraverso il servizio.

```
Terminale — bash — 80×24
bash
whiterabbit:tools alex$ ./adb shell
# logcat
I/DEBUG   (   28): debuggerd: Nov 24 2009 14:48:45
I/vold    (   27): Android Volume Daemon version 2.0
I/vold    (   27): New MMC card 'SU02G' (serial 1012966) added @ /devices/platfo
rm/goldfish_mmc.0/mmc_host/mmc0/mmc0:e118
I/vold    (   27): Disk (blkdev 179:0), 1048576 secs (512 MB) 0 partitions
I/vold    (   27): New blkdev 179.0 on media SU02G, media path /devices/platform
/goldfish_mmc.0/mmc_host/mmc0/mmc0:e118, Dpp 0
I/vold    (   27): Evaluating dev '/devices/platform/goldfish_mmc.0/mmc_host/mmc
0/mmc0:e118/block/mmcblk0' for mountable filesystems for '/sdcard'
I/vold    (   27): Aborting start of /sdcard (bootstrap = 1)
I/vold    (   27): Volmgr not ready to handle device
E/vold    (   27): Error opening switch name path '/sys/class/switch/test2' (No
such file or directory)
E/vold    (   27): Error bootstrapping switch '/sys/class/switch/test2' (No such
 file or directory)
E/vold    (   27): Error opening switch name path '/sys/class/switch/test' (No s
uch file or directory)
E/vold    (   27): Error bootstrapping switch '/sys/class/switch/test' (No such
file or directory)
D/vold    (   27): Bootstrapping complete
D/qemud   (   35): entering main loop
D/qemud   (   35): fdhandler_accept_event: accepting on fd 10
```

Figura 4.2: adb shell - logcat

Il tool ADB permette la connessione ad una *shell* remota, attraverso la quale è possibile agire come amministratore e visualizzare log, interrompere processi, modificare/cancellare/creare file. Un tool integrato nella *shell* di ambiente Android è "*logcat*", invocando tale comando nella *shell* remota è possibile visionare eventuali errori di sistema e/o di applicativi utente o nativi. Tale tool è risultato di notevole importanza durante tutto il processo di testing dell'applicativo permettendo di individuare e successivamente correggere errori che affliggevano il progetto.

4.3 Differenze tra Emulatore e dispositivo reale

Un dispositivo reale con sistema Android è pressoché equivalente al dispositivo virtuale avviato attraverso l'emulatore, tuttavia durante la fase di sperimentazione sono emerse differenze sostanziali tra le due piattaforme. E' importante premettere che l'emulatore è stato sempre avviato in ambiente Mac OSX 10.6 e i risultati riportati si riferiscono esclusivamente a tale ambiente. L'acquisizione audio dal microfono virtuale dell'emulatore è risultata più volte distorta a differenza dei test effettuati sul dispositivo reale. Durante le prove effettuate con l'acquisizione video è risultato inoltre che l'emulatore non si interfacciava correttamente con la webcam di sistema e la riproduzione video non risultava

fluida. Per tutte le prove di riproduzione/acquisizione video è stato necessario utilizzare il dispositivo reale.

Ecco di seguito una tabella riassuntiva:

Funzione	Simulatore	Dispositivo
Esecuzione da Eclipse	SI	SI
Console di Output	SI	SI
Debug con punti di interruzione	SI	SI
Riproduzione audio	SI	SI
Riproduzione video	NO	SI
Registrazione audio	SI	SI
Registrazione video	NO	SI
Accesso root	SI	NO
Modifica *filesystem* interno	SI	NO

4.4 Testing dell'applicativo

L'applicazione una volta avviata mostra l'interfaccia principale nella quale l'utente può inserire il numero della conferenza da contattare (dettaglio che dovrà essere rimosso nell'integrazione con il client esistente):

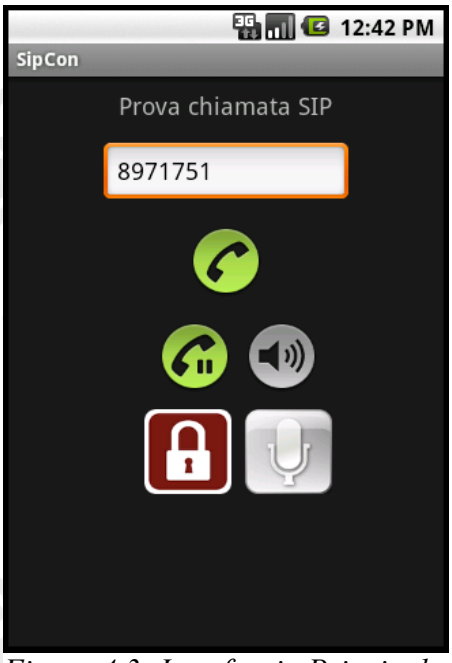


Figura 4.3: Interfaccia Principale

4.4.1 Avvio della comunicazione

Il primo pulsante di colore verde raffigurante una cornetta telefonica permette all'utente di avviare la comunicazione.

Successivamente dopo aver avviato la comunicazione e dopo aver ricevuto i dettagli del server BFCP l'utente potrà richiedere il permesso per iniziare la trasmissione del flusso audio in acquisizione, attraverso l'apposito pulsante raffigurante un microfono sbarrato da una linea diagonale di colore rosso. Come si può notare dall'immagine seguente l'utente è anche avvisato da una notifica testuale:

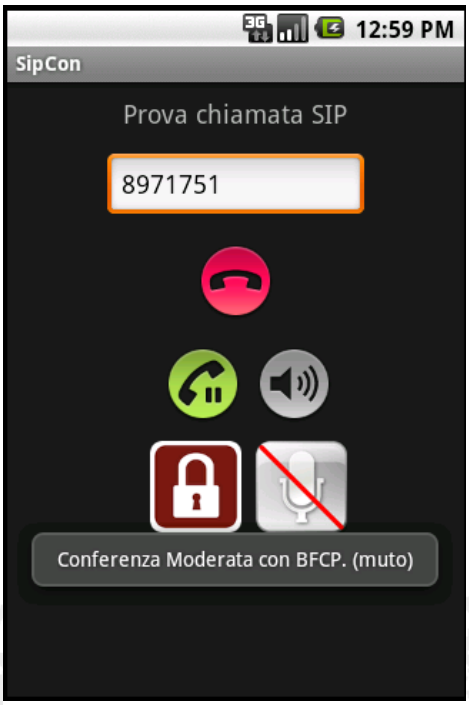


Figura 4.4: Dettagli BFCP ricevuti

Durante questa fase analizzando il traffico con il tool Wireshark, che consente di catturare e visualizzare il contenuto dei pacchetti trasmessi da un'interfaccia di rete, è possibile verificare che l'applicativo ha già avviato la fase di registrazione SIP attraverso l'uso del modulo `UserAgent`.

Nel seguente output di Wireshak i messaggi inviati dal client sono di colore rosso mentre quelli inviati dal server sono di colore blu:

```
REGISTER sip:1.2.3.4 SIP/2.0
Via: SIP/2.0/UDP 10.0.2.15:45304;rport;branch=z9hG4bK11921
Max-Forwards: 70
To: <sip:guest2@1.2.3.4>
From: <sip:guest2@1.2.3.4>;tag=z9hG4bK04730768
Call-ID: 590404572610@10.0.2.15
CSeq: 1 REGISTER
Contact: <sip:guest2@10.0.2.15:45304;transport=udp>
Expires: 3600
User-Agent: AndroEcho/0.1/sdk
Content-Length: 0

SIP/2.0 100 Trying
[...]
SIP/2.0 200 OK
[...]
```

Successivamente, come si può notare dal seguente output di Wireshark, viene avviata la comunicazione dove si nota il *three-way-handshaking* tra client e server per la contrattazione dei *codec* da utilizzare durante lo scambio dei flussi multimediali. Nel primo messaggio di *invite* il client comunica i propri *codec*, successivamente il server risponderà con la lista dei propri, il client infine risponderà con un messaggio di `ACK` confermando il messaggio ricevuto.

```
INVITE sip:8971751@1.2.3.4 SIP/2.0
Via: SIP/2.0/UDP 10.0.2.15:45304;rport;branch=z9hG4bK66630
[...]
Content-Type: application/sdp
v=0
o=guest2@1.2.3.4 0 0 IN IP4 10.0.2.15
s=Session SIP/SDP
c=IN IP4 10.0.2.15
t=0 0
m=audio 21000 RTP/AVP 8 0 3 101
a=rtpmap:8 PCMA/8000
a=rtpmap:0 PCMU/8000
a=rtpmap:3 GSM/8000
a=rtpmap:101 telephone-event/8000
a=fmtp:101 0-15

SIP/2.0 100 Trying
[...]
SIP/2.0 200 OK
Via: SIP/2.0/UDP 10.0.2.15:45304;branch=z9hG4bK66630;received=1.2.3.4;rport=61426
[...]
Content-Type: application/sdp
Content-Length: 343
v=0
o=root 12618 12618 IN IP4 1.2.3.4
s=session
c=IN IP4 1.2.3.4
b=CT:384
t=0 0
m=audio 10452 RTP/AVP 3 0 8 101
a=rtpmap:3 GSM/8000
a=rtpmap:0 PCMU/8000
a=rtpmap:8 PCMA/8000
a=rtpmap:101 telephone-event/8000
a=fmtp:101 0-16
a=ptime:20
a=sendrecv

ACK sip:8971751@1.2.3.4 SIP/2.0
[...]
```

Successivamente il server invia un messaggio di *re-invite* modificando la comunicazione attuale, introducendo i dettagli necessari al modulo `UserAgent` per la corretta connessione al server BFCP. In tale messaggio sono specificati il porto su cui è in ascolto il server BFCP, l'id dell'utente e l'id della conferenza, oltre alla corrispondenza tra gli id dei *floor* e i rispettivi *mediastream*.

```
INVITE sip:guest2@10.0.2.15:45304;transport=udp SIP/2.0
[...]
Content-Type: application/sdp
Content-Length: 671
v=0
o=root 12618 12619 IN IP4 1.2.3.4
s=session
c=IN IP4 1.2.3.4
b=CT:384
t=0 0
m=audio 10452 RTP/AVP 3 0 8 101
a=rtpmap:3 GSM/8000
a=rtpmap:0 PCMU/8000
a=rtpmap:8 PCMA/8000
a=rtpmap:101 telephone-event/8000
a=fmtp:101 0-16
a=ptime:20
a=label:10
a=sendrecv
m=application 2345 TCP/BFCP *
a=setup:passive
a=connection:new
a=floorctrl:s-only
a=confid:8971751
a=userid:133
a=floorid:11 m-stream:10
a=floorid:22 m-stream:11
a=floorid:33 m-stream:12

SIP/2.0 100 Trying
[...]

SIP/2.0 200 OK
Via: SIP/2.0/UDP 1.2.3.4:5060;branch=z9hG4bK31be63fc;rport=5060
[...]
Content-Length: 321
Content-Type: application/sdp
v=0
o=root 12618 12619 IN IP4 1.2.3.4
s=session
c=IN IP4 10.0.2.15
t=0 0
m=audio 21000 RTP/AVP 8 0 3 101
a=rtpmap:3 GSM/8000
a=rtpmap:0 PCMU/8000
a=rtpmap:8 PCMA/8000
a=rtpmap:101 telephone-event/8000
m=application 9 TCP/BFCP *
c=IN IP4 10.0.2.15
a=setup:active
a=connection:new
a=floorctrl:c-only

ACK sip:guest2@10.0.2.15:45304;transport=udp SIP/2.0
[...]
```

Le fasi precedenti sono confermate anche dall'output ottenuto tramite il comando "*logcat*":

```
I/System.out(  196): BFCP: received hello for conference: 8971751, userid: 167
D/AudioHardware(   31): AudioStreamInGeneric::read(0x40409180, 320) from fd 7
D/AudioHardware(   31): AudioStreamInGeneric destructor
I/System.out(  196): SipProvider: received new SIP message INVITE
sip:guest2@10.0.2.15:49179;transport=udp SIP/2.0
I/System.out(  196): ********* Microfono OFF!!
I/NotificationService(   52): enqueueToast pkg=com.sipcon
callback=android.app.ITransientNotification$Stub$Proxy@437b8c68 duration=0
I/System.out(  196): SipProvider: received new SIP message ACK
sip:guest2@10.0.2.15:49179;transport=udp SIP/2.0
I/System.out(  196): UA: CONFIRMED/CALL
```

L'utente potrà decidere di terminare la comunicazione corrente ripremendo lo stesso pulsante.

4.4.2 Richiesta risorsa tramite BFCP

L'utente dopo aver premuto il pulsante raffigurante il microfono, resterà in attesa che il moderatore dia il consenso per la concessione della risorsa. L'ottenimento della risorsa sarà notificato all'utente attraverso il cambio di stato dell'icona e con apposite notifiche testuali, come si può notare nella figura seguente:

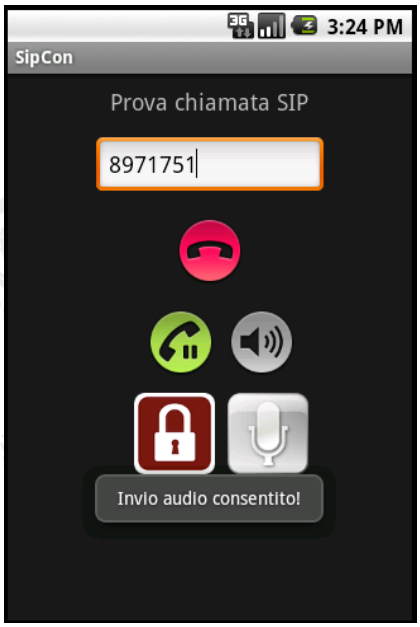


Figura 4.5: BFCP invio audio

L'utente sarà quindi in grado di partecipare attivamente alla conferenza dopo aver ottenuto il permesso dal moderatore della conferenza.

4.4.3 Conferenza in pausa

L'utente potrà inoltre mettere in pausa la trasmissione dei flussi multimediali premendo l'apposito pulsante di colore verde durante la comunicazione, il risultato prodotto sarà visionabile attraverso il comando "*logcat*" della *shell* remota:

```
I/System.out(  196): UA: RE-INVITING/MODIFYING
I/System.out(  196): UA: REFER/TRANSFER
I/NotificationService(   52): enqueueToast pkg=com.sipcon
callback=android.app.ITransientNotification$Stub$Proxy@4391e5b8 duration=0
D/dalvikvm(  196): GC freed 17171 objects / 507368 bytes in 150ms
I/System.out(  196): SipProvider: received new SIP message null SIP/2.0 100 Trying
I/System.out(  196): SipProvider: received new SIP message null SIP/2.0 200 OK
I/System.out(  196): UA: RE-INVITE-ACCEPTED/CALL
```

Il risultato che tale azione ha sull'interfaccia grafica principale è mostrato nella seguente figura, dove notiamo il cambio di stato del pulsante premuto dall'utente e la notifica testuale comparsa a video.

Figura 4.6: Conferenza in attesa

L'utente potrà decidere di ritornare alla conferenza ripremendo lo stesso pulsante.

4.4.4 Riproduzione Audio in vivavoce

L'utente potrà decidere di riprodurre la conferenza in vivavoce premendo l'apposito pulsante di colore grigio raffigurante un altoparlante. Alla pressione di tale pulsante l'utente sarà notificato con il cambio di stato dell'icona e con una relativa notifica testuale:

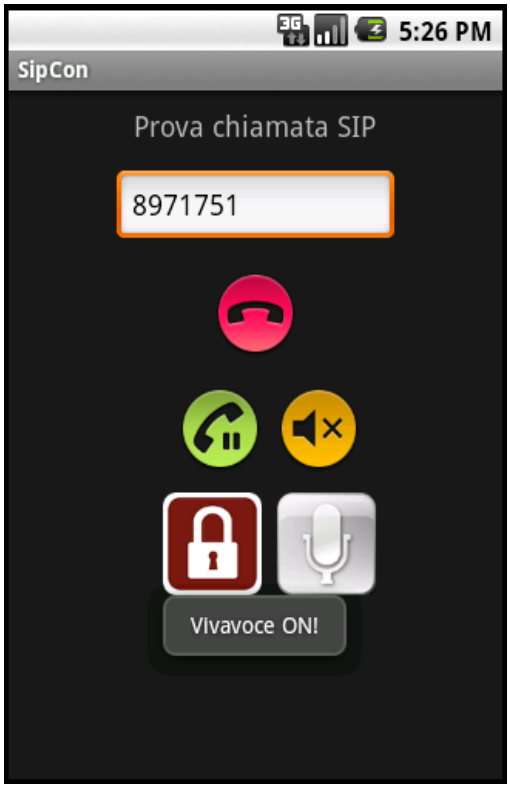


Figura 4.7: Vivavoce attivato

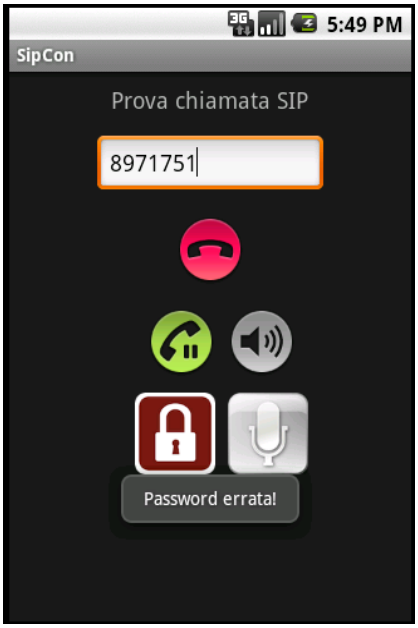
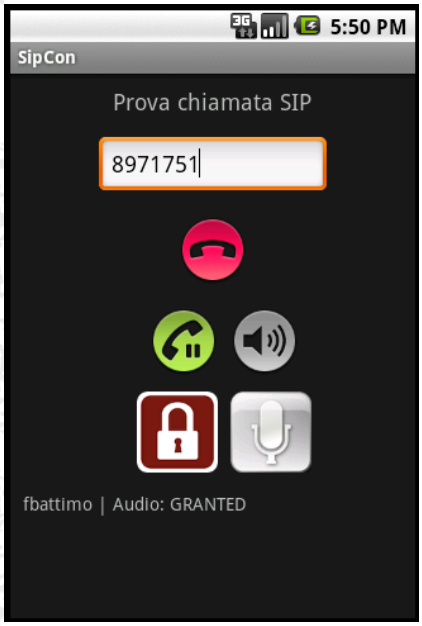
L'utente potrà decidere di ritornare alla modalità standard di riproduzione ripremendo lo stesso pulsante.

4.4.5 Menu moderatore

L'utente della conferenza può inoltre scegliere di diventare moderatore se è in possesso della password associata a quella conferenza. Premendo l'apposito pulsante di colore rosso raffigurante un catenaccio, sarà visualizzato all'utente un menu che gli permetterà di scegliere le risorse di cui può ottenere la moderazione e un apposito campo per l'inserimento della chiave segreta associata a quella determinata conferenza, come si può notare nella figura a lato.

Figura 4.8: Menu moderatore

Di seguito sono mostrati i casi di inserimento di password corretta e errata:

In caso di password errata l'utente sarà avvertito mediante una notifica testuale:  *Figura 4.9: Password Errata*	In caso di password corretta l'utente sarà avvertito mediante una notifica testuale e sarà visualizzata la lista degli utenti aventi risorse garantite:  *Figura 4.10:Password Corretta*

Il moderatore sarà notificato di un'eventuale richiesta di risorsa da parte di un utente attraverso una finestra a comparsa che permetterà la scelta dell'azione da compiere, come mostrato nella figura di seguito:

Figura 4.11: Richiesta risorsa

Il moderatore della conferenza premendo a lungo su un elemento della lista degli utenti con risorse garantite/pendenti può decidere di revocare risorse oppure accettare/negare la richiesta di acquisizione risorsa in attesa, attraverso una finestra a comparsa che permetterà la scelta dell'azione da compiere, come mostrato nella figura di seguito:

Figura 4.12: Azione moderatore

4.4.6 Interruzione della comunicazione

Durante una comunicazione attiva alla pressione del pulsante di colore rosso raffigurante una cornetta, la comunicazione viene interrotta e tutti i pulsanti che avevano subito un cambio di stato vengono riportati al loro stato originario.

Comunicazione Attiva:

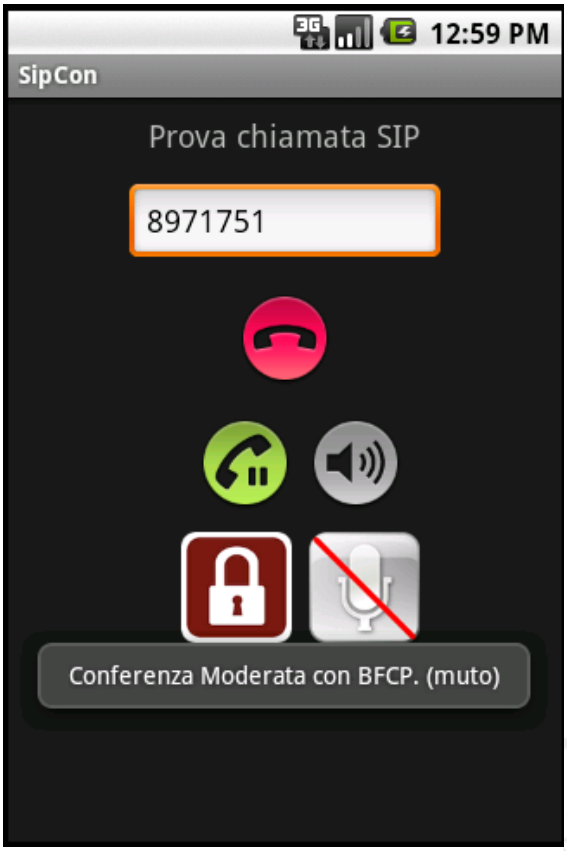


Figura 4.13: Comunicazione attiva

Comunicazione Terminata:

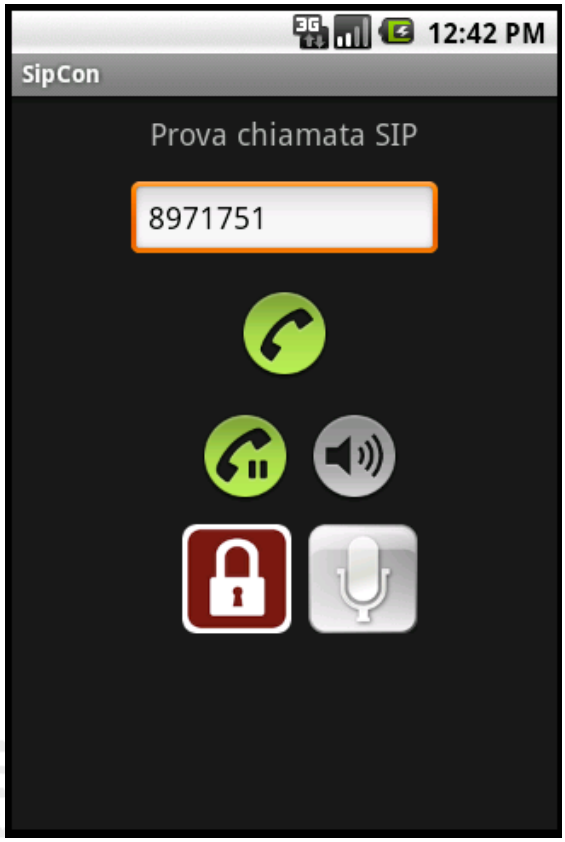


Figura 4.14: Comunicazione terminata

Tale processo può essere verificato anche attraverso l'output ottenuto dalla *shell* remota invocando il comando "*logcat*":

```
I/System.out(  196): UA: HANGUP
I/System.out(  196): AudioLauncher: halting java audio..
I/System.out(  196): running = false
I/System.out(  196): call.hangup()
D/AudioHardware(   31): AudioStreamInGeneric destructor
I/System.out(  196): RtpStreamSender: rtp sender terminated
I/System.out(  196): SipProvider: received new SIP message null SIP/2.0 200 OK
I/System.out(  196): Program will now exit
I/System.out(  196): UA: CLOSE/OK
I/System.out(  196): RtpStreamReceiver: rtp receiver terminated
I/System.out(  196): RegisterAgent: Unregistering contact
<sip:guest2@10.0.2.15:49179;transport=udp>
```

4.5 Testing sul dispositivo reale

L'applicativo implementato è stato testato con successo su dispositivi: HTC Tattoo e Acer Liquid. Il dispositivo HTC Tattoo è un telefono cellulare dotato di sistema Android *1.6*, il dispositivo Acer Liquid è invece dotato di sistema Android *2.1*. Il testing su dispositivi reali è stato di fondamentale importanza date le forti limitazioni dovute all'uso dell'emulatore. Entrambi i dispositivi infatti non sono affetti da problemi di distorsione durante l'acquisizione audio, presentati invece dall'emulatore. Ecco di seguito un'immagine che ritrae il dispositivo Acer Liquid con in esecuzione l'applicativo sviluppato:

Figura 4.15: Dispositivo Acer Liquid - Muto

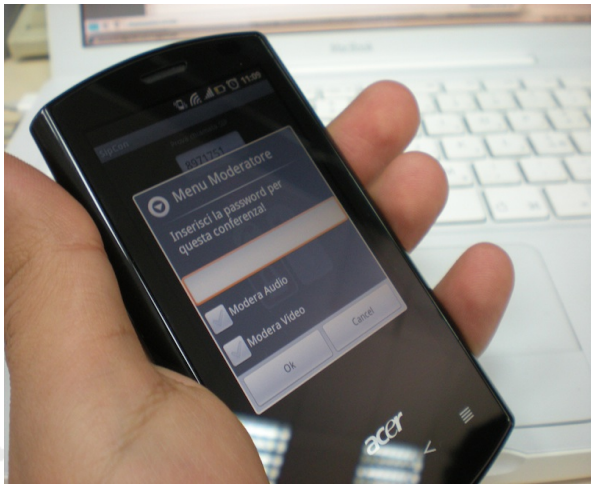


Figura 4.16: Acer Liquid - Menu Moderatore

4.6 Testing delle prestazioni

Le prestazioni sul dispositivo e sull'emulatore sono state analizzate attraverso il *tool* nativo di monitoraggio ***top***: tale strumento ha permesso la verifica delle risorse utilizzate dall'applicativo.

Per l'analisi delle prestazioni si sono individuati tre scenari differenti nei quali l'applicativo compiva azioni elaborative differenti:

- **Scenario A**: Ricezione audio, il *client* riceve solo i flussi multimediali audio e non li trasmette poiché non ha ancora effettuato richiesta della risorsa moderata.
- **Scenario B**: Trasmissione e Ricezione audio, il client ha effettuato la richiesta di risorsa moderata e si appresta a trasmettere i flussi multimediali acquisiti.

- **Scenario C**: Comunicazione in pausa, il client effettua un *re-invite* mostrando il suo disinteressamento alla ricezione dei media richiesti e interrompe l'acquisizione dei flussi multimediali locali.

Di seguito sono proposte alcune schermate registrate durante l'acquisizione dei dati di test:

```
Terminale — adb — 86×26
adb

User 7%, System 21%, IOW 0%, IRQ 1%
User 23 + Nice 0 + Sys 68 + Idle 214 + IOW 0 + IRQ 0 + SIRQ 5 = 310

  PID CPU% S  #THR     VSS     RSS PCY UID      Name
10494   9% S   12 171260K  22384K  fg app_82   com.sipcon
10551   7% R    1    876K    396K  fg root     top
   60   6% S    9  37680K   2204K  fg media    /system/bin/mediaserver
 9675   4% S    1      0K      0K  fg root     dhd_dpc
 9677   3% S    1      0K      0K  fg root     ksdioirqd/mmc1
   68   2% S    4   5408K    468K  fg compass  /system/bin/sensorserver_yamaha
  569   2% S   61 297596K  43168K  fg system   system_server
  991   0% S   13 165540K  13744K  bg app_98   com.geekyouup.android.widgets.battery
    5   0% S    1      0K      0K  fg root     events/0
 9678   0% S    1   1900K    456K  fg wifi     /system/bin/wpa_supplicant
   12   0% S    1      0K      0K  fg root     modem_notifier
   13   0% S    1      0K      0K  fg root     smd_tty
   14   0% S    1      0K      0K  fg root     qmi
   15   0% S    1      0K      0K  fg root     ctl0
   16   0% S    1      0K      0K  fg root     ctl1
   17   0% S    1      0K      0K  fg root     ctl2
   18   0% S    1      0K      0K  fg root     nmea
   19   0% S    1      0K      0K  fg root     rpcrouter
   20   0% D    1      0K      0K  fg root     rpcrotuer_smd_x
   21   0% S    1      0K      0K  fg root     krpcserversd
```

Figura 4.17: top – Scenario B

```
Terminale — adb — 86×26
adb

User 2%, System 11%, IOW 0%, IRQ 0%
User 9 + Nice 0 + Sys 39 + Idle 281 + IOW 0 + IRQ 0 + SIRQ 0 = 329

  PID CPU% S  #THR     VSS     RSS PCY UID      Name
10618   6% R    1    876K    396K  fg root     top
  569   4% S   61 297596K  43284K  fg system   system_server
 9677   1% S    1      0K      0K  fg root     ksdioirqd/mmc1
10494   1% S   13 172328K  22600K  fg app_82   com.sipcon
   57   0% S   11  17244K   1464K  fg radio    /system/bin/rild
 3114   0% S    5   4544K    340K  fg root     /sbin/adbd
    7   0% S    1      0K      0K  fg root     suspend
    8   0% S    1      0K      0K  fg root     kblockd/0
    9   0% S    1      0K      0K  fg root     kmmcd
   10   0% S    1      0K      0K  fg root     btaddconn
   11   0% S    1      0K      0K  fg root     btdelconn
   12   0% S    1      0K      0K  fg root     modem_notifier
   13   0% S    1      0K      0K  fg root     smd_tty
   14   0% S    1      0K      0K  fg root     qmi
   15   0% S    1      0K      0K  fg root     ctl0
   16   0% S    1      0K      0K  fg root     ctl1
   17   0% S    1      0K      0K  fg root     ctl2
   18   0% S    1      0K      0K  fg root     nmea
   19   0% S    1      0K      0K  fg root     rpcrouter
   20   0% D    1      0K      0K  fg root     rpcrotuer_smd_x
```

Figura 4.18: top – Scenario C

Nel processo di acquisizione/riproduzione audio interviene il servizio di sistema *mediaserver* che si fa carico dell'esecuzione di tali operazioni. È importante notare come nello scenario C riportato nella figura 4.18 non sia presente tale *mediaserver* poiché la comunicazione è in pausa e non ci sono flussi multimediali da acquisire/riprodurre.

Di notevole interesse è la differenza di carico della CPU tra i tre scenari proposti. Analizzando il grafico riportato in figura 4.19, c'è un notevole incremento del carico, dal

passaggio dallo scenario *A* allo scenario *B*, dovuto all'avvio dell'acquisizione dei flussi multimediali audio locali da inviare al server SIP. Analizzando poi lo scenario C, come ci si aspettava, c'è un notevole decremento del carico della CPU, dovuto dal fatto che la comunicazione è in pausa e quindi il client non si occupa della pacchettizzazione/de-pacchettizzazione e relativa acquisizione/riproduzione dei flussi multimediali.

In entrambi i grafici le prestazioni su emulatore e dispositivo si equivalgono.

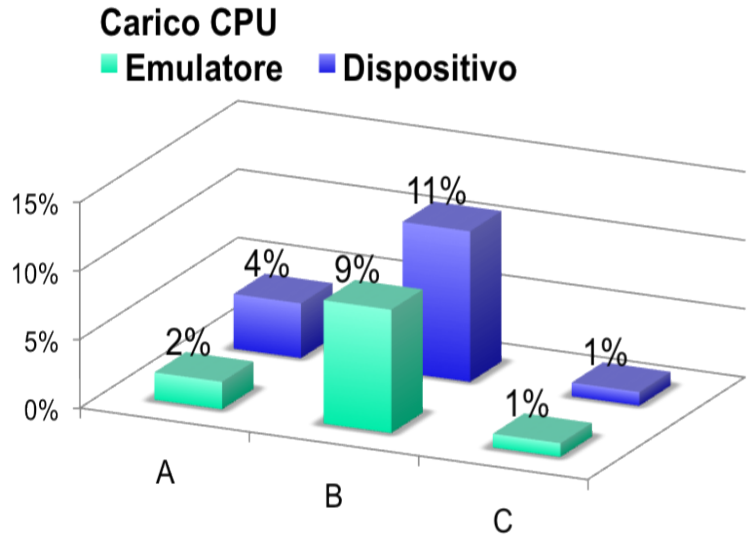


Figura 4.19: Grafico Carico CPU

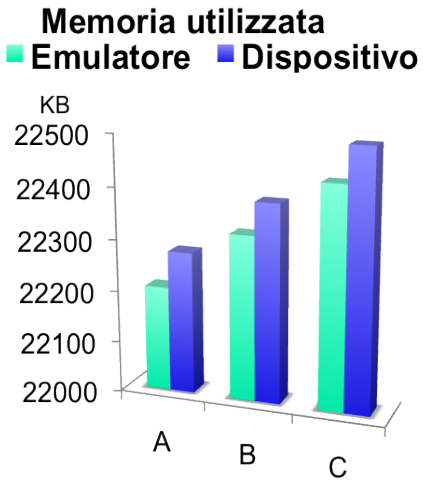


Figura 4.20: Grafico Memoria

Analizzando il grafico riportato in figura 4.20, la memoria utilizzata risulta relativamente costante nel tempo.

4.7 Problemi riscontrati

La gran parte dei problemi riscontrati sono stati evidenziati durante la fase di sperimentazione, durante tale processo infatti sono emersi problemi relativi alla riproduzione di *buffer* di flussi multimediali di tipo video, causando l'impossibilità di riproduzione dei flussi multimediali video ricevuti dal server SIP.

Le API (*Application Programming Interface*) dell'ambiente Android forniscono la sola classe `MediaPlayer` per la riproduzione video. Tale classe non supporta la riproduzione a partire da *buffer* multimediali, pur supportando i *codec* H.263 e MPEG4, né fornisce delle librerie per l'accesso diretto ai *codec*.

4.7.1 Soluzione con l'uso della classe nativa `MediaPlayer`

L'unica strada da seguire volendo utilizzare la classe nativa `MediaPlayer` era quella di trovare un modo di passare il *buffer* del flusso multimediale video ricevuto sotto forma di `ParcelFileDescriptor`. Infatti il metodo della classe `MediaPlayer` che permette il settaggio del file multimediale da riprodurre, oltre alla stringa contenente il percorso del file, accetta come argomento un oggetto di tipo `ParcelFileDescriptor`, cioè un descrittore di file.

Per l'acquisizione video si è riusciti a sfruttare l'utilizzo delle classi native dell'ambiente Android: `LocalSocket` e `LocalServerSocket`, dato che istanziando un oggetto di tipo `LocalSocket` è possibile invocare su tale istanza il metodo `getFileDescriptor()` che fornisce un'oggetto di tipo `ParcelFileDescriptor` a partire dal contenuto dei buffer `OutpurStream` e `InputStream` dell'istanza di tale classe. Si utilizzano quindi le due classi `LocalSocket` e `LocalServerSocket` per creare, attraverso la comunicazione delle due istanze delle classi, un buffer che potrà essere utilizzato per la lettura/scrittura dei flussi multimediali da far processare all'istanza della classe `MediaRecorder`, attraverso l'uso dell'oggetto di tipo `ParcelFileDescriptor`.

Per la riproduzione video invece, con l'uso della classe `MediaPlayer`, l'utilizzo di questo tunnel per i *buffer* ricevuti dal server SIP non ha avuto successo. Durante l'uso del metodo precedente infatti si è più volte riscontrato un errore di questo tipo:

```
E/MediaPlayerService(   31): offset error
E/MediaPlayer(  964): Unable to to create media player
W/System.err(  964): java.io.IOException: setDataSourceFD failed.: status=0x80000000
W/System.err(  964):   at android.media.MediaPlayer.setDataSource(Native Method)
W/System.err(  964):   at android.media.MediaPlayer.setDataSource(MediaPlayer.java:643)
W/System.err(  964):   at com.sockstream.sockStream$1.onClick(sockStream.java:41)
W/System.err(  964):   at android.view.View.performClick(View.java:2344)
W/System.err(  964):   at android.view.View.onTouchEvent(View.java:4133)
W/System.err(  964):   at android.widget.TextView.onTouchEvent(TextView.java:6510)
W/System.err(  964):   at android.view.View.dispatchTouchEvent(View.java:3672)
[...]
```

Si è giunti dunque alla conclusione che tale metodo risulta funzionante solo in scrittura attraverso l'uso della classe `MediaRecorder` ma non in lettura con l'uso della classe `MediaPlayer`.

4.7.2 Soluzione con l'uso di una implementazione Java del *codec* H.263

Una strada alternativa che è stata seguita per cercare di risolvere tale problema è stata quella di utilizzare un'implementazione Java del *codec* video H.263, frutto di un lavoro di tesi precedente.

Nell'uso di tale libreria si individuano tre oggetti: il `buffer`, il `decoder`, il `imageViewer`. Dopo aver estratto i dati dai pacchetti RTP ricevuti dalla rete, vengono passati all'oggetto `buffer` tramite uno *stream*; il `buffer` bufferizza i dati e li passa all'oggetto `decoder` tramite un altro *stream*; il `decoder` converte i dati video in una sequenza di immagini, codificate in RGB (*Red-Green-Blue*), e le passa all'oggetto `imageViewer` tramite l'interfaccia `H263DecoderListener`; il `imageViewer` infine effettua il "*rendering*" del video, cioè visualizza le immagini sul display.

Di seguito è mostrato un diagramma delle classe che illustra le associazioni tra i moduli:

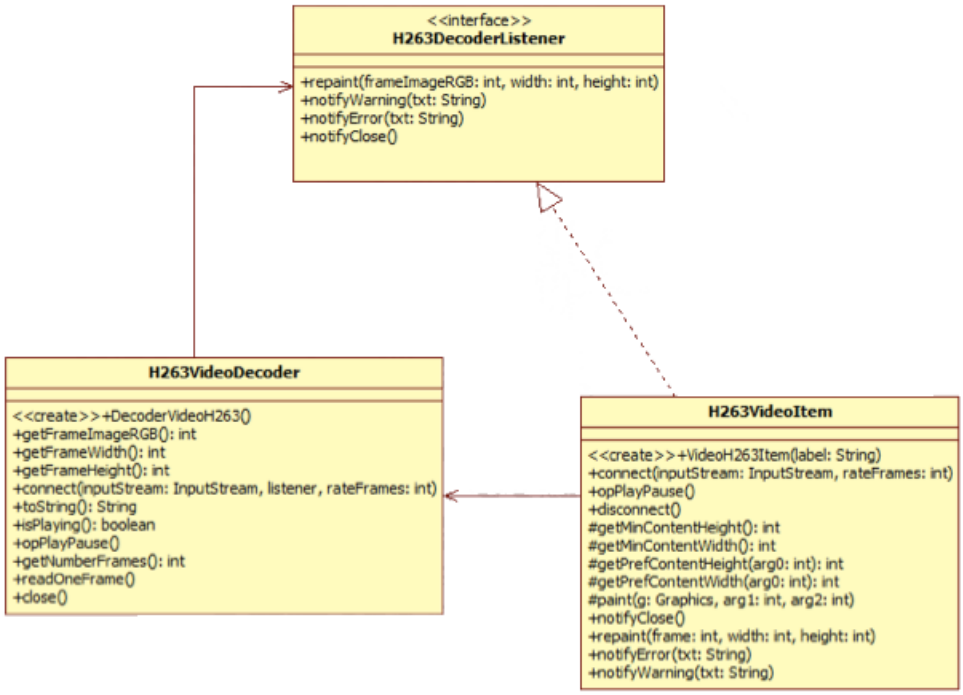


Figura 4.21: Diagramma delle classi libreria codec H.263

L'uso di tale libreria implementata per ambiente Java SE non ha però portato risultati tangibili dato che non decodificava correttamente i dati sottoposti. Pertanto il suo utilizzo è stato abbandonato, non si esclude però che uno studio approfondito dei problemi riportati porti ad un possibile utilizzo della libreria.

4.7.3 Soluzione con l'uso di un server RTSP

L'ultimo tentativo effettuato per cercare di permettere al dispositivo di riprodurre correttamente il flusso multimediale video inviato dal server SIP è stato quello di simulare un server RTSP (*Real Time Streaming Protocol*) lato client. Questa soluzione è stata preferita alle altre perché risultata funzionante nell'applicativo Sipdroid ma soggetta ad eccessivi ritardi. Al giorno d'oggi qualsiasi dispositivo mobile di ultima generazione è dotato di un *Mediaplayer* in grado di riprodurre contenuti multimediali in streaming attraverso l'uso del protocollo RTSP.

Per chiarire meglio tale soluzione è proposto un diagramma dei moduli coinvolti:

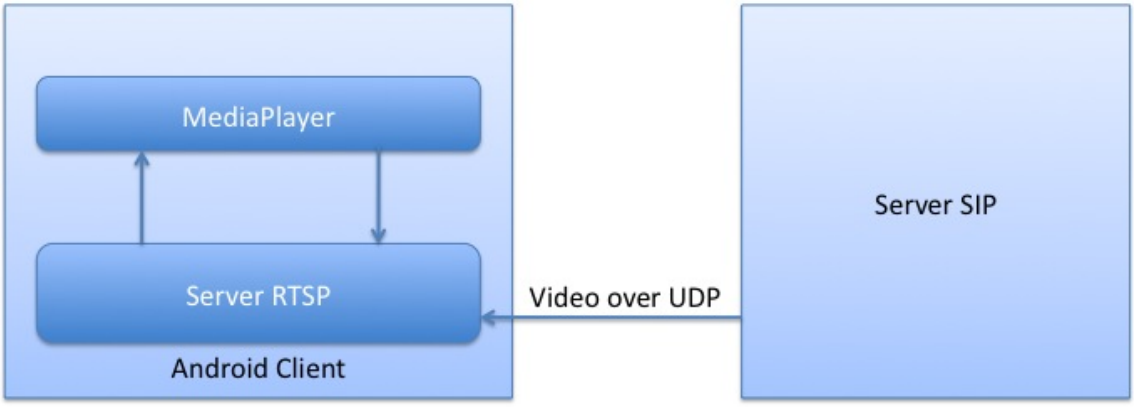


Figura 4.22: Diagramma dei moduli nella soluzione "Server RTSP"

Tale strategia prevede tutti i passaggi precedentemente descritti per il normale collegamento ad una conferenza. All'atto del collegamento sarà richiesto però anche la risorsa video in modo da permettere la ricezione di tale flusso multimediale. Il server RTSP, posto lato client, però non può notificare al server SIP l'invio dei flussi multimediali direttamente al *MediaPlayer* poiché all'atto della richiesta il MediaPlayer interno avrà già acquisito le *socket* sui rispettivi porti attraverso le quali si appresterà a ricevere tale flusso multimediale. Pertanto l'applicativo Android non potrà più utilizzare tali porti per iniziare la comunicazione e permettere al server SIP di inviare su essi i flussi multimediali richiesti.

Quindi il server RTSP si deve far carico della trasmissione/ricezione di flussi multimediali

e smistare tali flussi al MediaPlayer attraverso delle *socket* locali. Il tutto può essere riassunto attraverso il seguente *Sequence Diagram*:

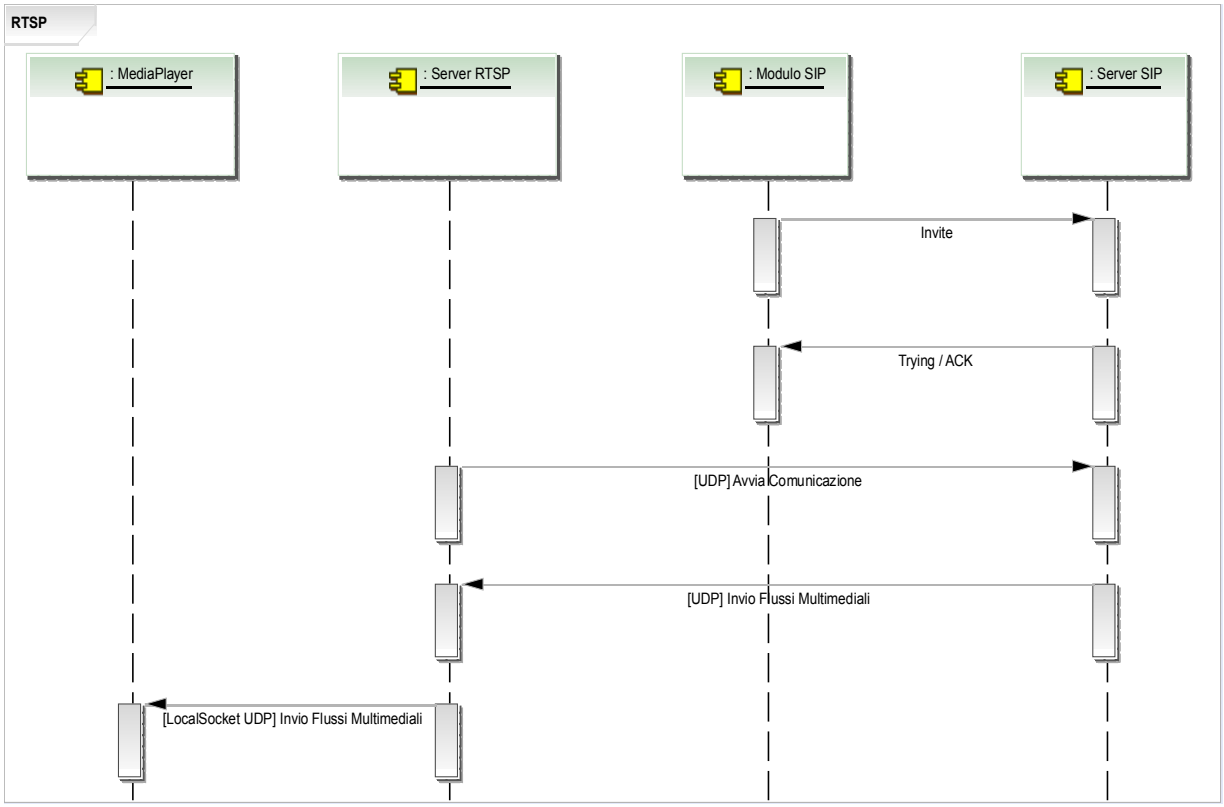


Figura 4.23: Sequence Diagram Modulo Server RTSP

Tale strategia ha dato risultati soddisfacenti, tuttavia il ritardo che si otteneva non era accettabile per comunicazioni in *realtime*. Tale soluzione necessita quindi di ulteriori studi al fine di ridurre il ritardo introdotto dall'uso di tale protocollo.

Conclusioni e Sviluppi Futuri

In questo capitolo sarà riepilogato il lavoro fatto con un accenno agli sviluppi futuri.

Conclusioni

L'obiettivo di tale lavoro di tesi è stato compiuto portando a termine l'implementazione di un applicativo in grado di interfacciarsi con la piattaforma *Meetecho* e capace di scambiare con tale piattaforma contenuti multimediali attraverso i seguenti protocolli:

- SIP (*Session Initiation Protocol*)
- SDP (*Session Descriptor Protocol*)
- BFCP (*Binary Floor Control Protocol*)
- RTP (*Real Time Protocol*)

La prima fase di tale lavoro di tesi si è concentrata sullo studio della piattaforma Android, tale studio ha comportato l'apprendimento delle caratteristiche architetturali della piattaforma e l'apprendimento delle caratteristiche gerarchiche dell'interfaccia grafica in ambiente Android.

Successivamente lo studio si è spostato sui *client* di *conferencing* già esistenti in ambiente Android, andando ad esaminare ove possibile le caratteristiche implementate e le limitazioni introdotte da tali applicativi.

Si è quindi passati alla progettazione di tale applicativo tenendo conto della piattaforma Meetecho con cui interfacciarsi e delle eventuali limitazioni introdotte dal dispositivo.

Tale applicativo è stato progettato in modo da astrarre i protocolli di rete dall'interfaccia grafica che li utilizza attraverso un modulo intermedio. È stato inoltre progettato per essere esteso da ulteriori funzionalità e integrato in un client frutto di un lavoro di tesi precedente. Durante la fase di implementazione si è deciso di riutilizzare in parte alcune librerie esistenti, open source o frutto di lavori di tesi precedenti. Tale soluzione ha portato alla creazione di un software modulare in grado di essere ampliato o aggiornato nei suoi

singoli moduli senza particolari problemi. L'implementazione non si è fermata alle sole funzionalità richieste per il corretto interfacciamento con la piattaforma Meetecho, tale applicativo infatti fornisce la possibilità all'utente finale di:

- Collegarsi ad una conferenza
- Richiedere/Rilasciare una risorsa moderata
- Passare alla modalità di comunicazione vivavoce
- Mettere in pausa la comunicazione
- Autenticarsi come moderatore della conferenza

L'implementazione dell'applicativo è stata portata a termine attraverso l'uso del linguaggio di programmazione Java utilizzato con il rispettivo *plugin* del tool di sviluppo Eclipse.

La fase finale è stata incentrata sulla sperimentazione e lo studio delle prestazioni dell'applicativo sviluppato. La sperimentazione, sia sull'emulatore che sul dispositivo reale è stata di fondamentale importanza per la correzione di eventuali problematiche non riscontrabili in fase implementativa.

Le problematiche maggiori si sono riscontrate nell'implementazione del modulo necessario alla comunicazione video e necessitano ancora di una fase di studio più approfondita che permetta lo sviluppo di una soluzione adeguata.

Si può quindi affermare che l'applicativo sviluppato integrato nel client XMPP (*eXtensible Messaging and Precence Protocol*), frutto di un lavoro di tesi precedente, costituisce una soluzione completa per il *conferencing* testuale e audio attraverso la piattaforma Meetecho.

Sviluppi Futuri

In primo luogo è fondamentale in un futuro prossimo lavorare alla visualizzazione di un flusso multimediale video ricevuto, in tal campo si sono mossi i primi passi di sperimentazione, ma lo studio deve essere approfondito scegliendo magari una delle strade proposte in questo lavoro di tesi.

Tutti i dispositivi Android fin ora messi in commercio non sono dotati di fotocamera frontale e non permettono la funzionalità di videochiamata tuttavia in uno scenario di videoconferenza potrebbe essere comunque utile fornire un flusso video acquisito con la fotocamera posteriore agli interlocutori.

Inoltre date le potenzialità della piattaforma Android sarebbe opportuno estendere tutte le altre funzionalità avanzate che la piattaforma Meetecho mette a disposizione per il client Desktop quali: lavagna condivisa, visualizzazione *desktop* remoto e così via..

È possibile dunque riassumere le funzionalità da implementare nell'applicativo frutto di questo lavoro di tesi nel seguente modo:

- Visualizzazione di un flusso multimediale video ricevuto
- Invio di un flusso multimediale video acquisito dalla fotocamera
- Servizi aggiuntivi (lavagna condivisa, *desktop* remoto)

Bibliografia

[1] Reto Meier 2010 "Professional Android 2 Development"

[2] Google Inc. 2010 "Android Developer Reference", http://developer.android.com/reference/packages.html

[3] Google Inc. 2010 "Android SDK", http://developer.android.com/sdk/index.html

[4] Google Inc. 2010 "Android Developer Tools", http://developer.android.com/guide/developing/tools/index.html

[5] The Sipdroid Open Source Project 2010 "Sipdroid client", http://www.sipdroid.org/

[6] Luca Veltri - University of Parma 2005 "MjSip – SIP Java Library", http://www.mjsip.org/

[7] Pasquale Di Rienzo 2010 "BFCP Control Center User Manual"

[8] Davide Cozzolino 2008 "Progetto e realizzazione di tecniche per il trasferimento di contenuti multimediali in tempo reale su dispositivi mobili"

[9] RTP - Real-time Transfer Protocol, http://www.ietf.org/rfc/rfc3550.txt

[10] SIP - Session Initiation Protocol http://www.ietf.org/rfc/rfc3261.txt

[11] SDP - Session Description Protocol http://www.ietf.org/rfc/rfc2327.txt

[11] BFCP - Binary Floor Control Protocol http://www.ietf.org/rfc/rfc4582.txt

[12] CCMP - Conference Control Manipulation Protocol http://tools.ietf.org/id/draft-ietf-xcon-ccmp-08.txt